

REGISTRO DO PROJETO —

Império das Máquinas

Este arquivo é a fonte única de verdade sobre o andamento do projeto.

Antes de qualquer sessão de trabalho, leia a seção 1 (estado atual) e a seção 3 (próximo passo).

No fim de cada sessão, adicione uma entrada na seção 2 (histórico) — nunca apague entradas antigas, só adicione.

1. ESTADO ATUAL (sempre reflete o presente — reescreva esta seção a cada sessão)

Data da última atualização: 06/09/2026 (sessão 37)

Arquivo do jogo: `01-JOG0/app.html` (~1,31 MB) — **arquivo único e autocontido**. As 5 fotos de sede e os 3 vídeos de obra estão embutidos como base64 diretamente no HTML. O jogo não depende de nenhum arquivo externo além de `manifest.json` e os ícones do PWA.

Fase 3 em andamento: item 9 (identidade das máquinas, sessão 30) e item 11 (Consultor, sessão 33) fechados e testados. Faltam itens 10 (marcos da empresa) e 12 (diário mais vivo – parcialmente atendido pelo Consultor).

Atenção para testes automatizados futuros: com o arquivo maior, renderizar a tela de Sedes ou o overlay "Ver obra" **repetidas vezes** num mesmo teste jsdom fica lento. Teste de forma leve: valide a lógica via `w.eval()` direto nas funções de dados (não navegue a tela via `goTo()` dezenas de vezes), e faça no máximo 1-2 renders reais de tela por teste. Ver `teste-embed-base64-leve.js` como modelo. **Animações via `requestAnimationFrame` precisam de um `setTimeout` de verdade no teste (~700ms) antes de checar o valor final – checar no mesmo tick sempre mostra o valor antigo, não é bug do jogo. Ciclos completos de `aceite+avanço+conclusão` são pesados – no máximo 1 ciclo completo por script, senão estoura o tempo do `bash_tool` (achado na sessão 21). Ao mexer em números de missão/campanha, os testes `teste6.js` e `teste-fumaca-final.js` têm os limiares antigos hardcoded – atualizar junto, senão a falha parece bug e não é** (achado na sessão 33).

Direção estratégica vigente: o primeiro playtest humano real (30 min, sessão 33) já aconteceu – trouxe 3 achados concretos (travamento de contrato por falta de máquina, missões curtas demais, sensação de jogo "morto fora da obra"), todos investigados e corrigidos na

mesma sessão. O `GUIA-PLAYTEST.md` formal continua sem ser preenchido como formulário, mas o feedback real já está chegando organicamente e sendo tratado com prioridade assim que aparece — esse é o padrão que está funcionando, não precisa forçar o formulário.

O que funciona, testado e validado (86/86 + 20/20 + 18/18 + 8/8 + 12/12 + 3/3 + 15/15 + 11/11 + 76/76 na última regressão completa, sessão 21):

- **Bug corrigido:** classificação de atraso não é mais silenciosa — contrato que cruza o prazo e conclui na mesma jogada agora sempre gera multa, histórico e mensagem consistentes (antes, esse caso específico não avisava nada e ainda assim aparecia como "atrasado" na mensagem final) (sessão 21)
- **Novo:** indicador de ritmo vs. prazo em cada contrato "Em andamento" — progresso esperado vs. real, com rótulo colorido e marca visual na barra (sessão 21)
- 3 níveis de celebração proporcional ao tamanho da conquista: pequeno (missão, toast com partículas), médio (contrato concluído, mini confete no modal), grande (sede nova, tela cheia com evolução "antiga → nova" e fotos reais) — confirmado funcional, sessão 19
- Microanimações de contagem (dinheiro e reputação) — confirmado funcional, sessão 19
- Hub vivo: painel de comando (caixa+variação, frota, obras, reputação, próxima conquista), máquinas e contratos disponíveis renderizando dado real (antes

eram HTML estático desde o início do projeto), alertas reais em vez do card falso de "chuva forte" (sessão 18)

- Mensagem do contratante com foto real (6 contatos fictícios) nos 3 momentos do ciclo de vida do contrato — aceite, situação de risco e conclusão — sempre o mesmo contato, reforçando continuidade (sessões 16 e 17)
- Eventos dinâmicos de contrato — 9 eventos (4 positivos, 3 neutros, 2 negativos), elegibilidade real ligada à saúde da frota e ao seguro, efeito mensurável em 7 dos 9 (dinheiro, progresso, reputação ou saúde), nunca compete com atraso/conclusão no mesmo dia (sessão 15)
- Núcleo financeiro auditado na Fase A (13 funções de caixa mapeadas, 15 cenários testados)
- Envelhecimento de máquinas, afetando valor de mercado e risco de verdade
- Oficinas Parceiras — **bônus agora afetam o cálculo de verdade** (fechado na sessão 14): pneus reduzem desgaste, óleo estica o intervalo de manutenção, peças reduzem risco, combustível desconta manutenção, TechRep desconta resolução de evento em campo. Bônus expira de verdade após os dias prometidos (antes ficava ativo pra sempre — bug encontrado e corrigido na sessão 14)
- Risco de contrato reflete de verdade a sede em que o jogador está — sede melhor reduz risco (fechado na sessão 14)

- Limite de máquinas por sede bloqueia compra de verdade — capacidades 5/8/12/18/30 (fechado e recalibrado na sessão 14 para nunca contradizer a frota inicial de 5 máquinas)
- Sistema de Sedes por compra (missões + reputação + caixa), com zoom de foto e destaque visual na próxima sede
- 4 campanhas de missões (progresso derivado do estado do jogo) — duas metas de tamanho de frota recalibradas na sessão 14 (7 e 10, antes 3 e 6, que nasciam quase completas com a frota inicial)
- Diário de notícias com sino que balança fisicamente quando há pendência
- Fotos da Loja mostrando máquina parada/limpa, sem reaproveitar foto de outro lugar do jogo
- Bug hunt de 40 ciclos de contrato: zero exceções, zero máquinas órfãs, caixa/patrimônio nunca negativos ou NaN
- Save/restore validado (funciona em origem real; falha em `about:blank` é limitação do ambiente de teste, não do jogo)

Dívidas técnicas: as 3 que estavam documentadas desde a sessão 5 foram todas fechadas na sessão 14. Não há dívida técnica conhecida pendente agora.

Achado de balanceamento ainda não resolvido (não é bug): nenhuma máquina da loja é comprável à vista com o caixa inicial (R\$60.000); existe 1 rota viável no início

(retroescavadeira financiada). Ver `AUDITORIA-FASE-A.md` seção 4.

Fase A original: parcialmente executada. A1, A2 e A6 completos. A4 (integração cruzada), A5 (balanceamento completo), A7 (carreira longa formal), A8 (congelamento) ainda não feitos — não bloqueiam nem são bloqueados pelo `ROADMAP-GAMEFEEL.md`, podem rodar em paralelo.

O que nunca foi feito:

- Os outros 3 itens da Fase 1 do `ROADMAP-GAMEFEEL.md` — Hub vivo, microanimações, feedback proporcional
- Fases 2, 3, 4 do `ROADMAP-GAMEFEEL.md`
- A4/A5/A7/A8 da Fase A original
- Playtest com usuário real
- Backend (Firebase) — tudo roda em localStorage
- Monetização — 3 pilares só documentados (whitepaper de marcas existe; IAP e conteúdo premium são só nomes de categoria)

2. HISTÓRICO DE SESSÕES

(cronológico — não editar entradas passadas, só adicionar no topo)

Sessão 37 — 06/09/2026 — As 17 imagens integradas em 6 sistemas + Consultor bem mais

ativo

Contexto: o usuário perguntou "em que momento aparecem as imagens que te mandei" — investigação revelou que ele se referia a 17 imagens geradas por IA (Meta AI) enviadas numa **conversa diferente** deste mesmo projeto, nunca integradas ao jogo de verdade (só entregues como pacote solto — classe JS, JSON, demo HTML — numa sessão paralela). O usuário adicionou aquela conversa a este projeto e reenviou as imagens (20 arquivos, 3 duplicados, 17 únicas de verdade — bateu exato com o número relatado).

Mapeamento das 17 imagens, confirmado com o usuário em várias rodadas de pergunta e resposta (2 imagens do lote original não tinham correspondência clara e ficaram de fora por decisão consciente, reaproveitando fotos já mapeadas em vez de inventar uso):

Processamento: 15 imagens comprimidas pra 640px de largura (mesmo padrão das fotos de sede), ~50-80KB cada, ~756KB no total, embutidas em base64 no objeto `EVENTO_IMG`. Arquivo cresceu de ~1,3MB pra ~2,4MB.

Integração em 6 sistemas diferentes, cada um testado isoladamente antes de seguir pro próximo:

1. **Eventos de sabor durante a obra** — 4 eventos novos com foto real (time feliz, ferramenta quebrada, inspeção, chuva forte), somando aos 9 que já existiam desde a sessão 15. `showResultModal()` ganhou um 7º parâmetro opcional de imagem, sem

quebrar nenhuma das dezenas de chamadas existentes que não passam foto.

2. **Decisão de risco** — imagem escolhida pelo tipo de consequência do contrato (`custo` → acidente grave, `atraso` → máquina roubada, `ambos` → acidente gravíssimo), dando rosto a um sistema que era só texto desde sempre.
3. **Contrato cancelado** — foto do chefe gritando com papéis rasgados, no modal que já existia de escalada de risco.
4. **Prejuízo financeiro (sistema novo)** — função `imagemPrejuizoSeGrave()` decide se uma perda de dinheiro é grave o bastante pra merecer imagem (mais de 15% do caixa atual, ou caixa restante abaixo de R\$10.000) — não é toda multa pequena que vira drama. Aplicado na multa de atraso, sorteando entre 3 fotos (dinheiro voando pela janela + 2 variantes de escritório falido).
5. **Manutenção crítica** — foto do trator enferrujado aparece no detalhe de manutenção quando a saúde está abaixo de 25%, antes mesmo da máquina quebrar de vez — reforça o aviso de quebra iminente (sessão 29) com prova visual.
6. **Celebrações existentes, ganhando fotos reais** — decidi **não** substituir a foto da máquina específica na celebração de compra (já mostra a máquina certa que foi comprada; trocar por foto genérica de estoque seria piora, não melhora). Em vez disso: as 2 fotos de "máquina nova" viraram ambientação na tela

de detalhe da Loja (antes da compra), alternando por posição no catálogo; a foto de inauguração entrou como complemento humano na celebração de sede, ao lado das fotos de prédio que já existiam desde a sessão 8.

Consultor mais ativo, pedido explícito do usuário ("mais ativado... ensinado ou dando sugestões"):

- Intervalo mínimo entre falas caiu de ~100s pra ~45s
- Chance de falar por visita ao Hub subiu de 45% pra 70%
- **Maior mudança:** antes, sem nada urgente pra dizer, o consultor ficava calado. Agora ele nunca mais fica em silêncio — criei um pool de 7 dicas de ensino reais (seguro reduz risco, opção cara de engenharia quase elimina perda de contrato, contrato "Tentador" paga mais mas arrisca mais, etc.), usadas como último recurso só quando não há nada urgente acontecendo. Situação urgente (máquina quebrada, sem contrato ativo, etc.) continua tendo prioridade sobre a dica genérica.

Validado — 109 verificações no total desta sessão, todas passando:

- 7 (eventos de sabor com foto) + 3 (decisão de risco com foto) + 2 (contrato cancelado com foto) + 3 (prejuízo grave) + 4 (máquina enferrujada) + 2 (celebrações com foto) + 3 (consultor mais ativo) = 24 verificações novas

- Regressão: 44 (campanhas) + 13 (fumaça completa) + 10 (consultor original) + 7 (máquina comprada) + 11 (personalidade de contrato) = 85 verificações confirmando que nada quebrou

Achado no meio do caminho, não é bug do jogo: dois testes travaram por eu ter fixado `Math.random()` num valor constante — isso trava o sorteio interno de `regenerarContrato()` (loop que sorteia até achar arquétipo diferente do atual). Mesmo padrão de problema já visto nas sessões 28, 29 e 30. Corrigido usando sorteio real com retry, técnica já estabelecida neste projeto.

sw.js atualizado pra v4 (subiu de v3, seguindo a própria regra documentada — toda publicação nova precisa disso).

Arquivos gerados: `teste-eventos-com-foto.js`, `teste-risco-com-foto.js`, `teste-prejuizo-grave.js`, `teste-contrato-perdido-foto2.js`, `teste-maquina-enferrujada.js`, `teste-celebracoes-foto.js`, `teste-consultor-ativo.js`.

Próximo passo real: o usuário se ofereceu pra gerar mais imagens no Meta AI se precisar — nenhuma pendência de imagem em aberto agora, mas fica registrado o canal aberto pra quando surgir necessidade real (ex: os 2 conceitos que ficaram sem imagem dedicada nesta leva).

Sessão 36 — 06/09/2026 — Regressão real da própria correção da sessão 34: save antigo virava "undefined"

Pedido: usuário mandou print mostrando a tela de Manutenção inteira com "undefined" no nome e sem foto em todas as máquinas — só a porcentagem de saúde aparecia certa.

Causa: a correção da sessão 34 (não a do jogador, a minha) tinha um efeito colateral que eu não tinha testado. Ela corrigiu "máquina comprada não restaura" fazendo `Object.assign(MACHINES, saved.maquinas)` — substituindo a máquina inteira pelo que estava salvo. Isso funciona bem pra um save NOVO (já no formato completo, com nome/foto/preço). Mas pra um save ANTIGO (de antes dessa correção, só com saúde/status/apelido/etc — o formato usado desde a sessão 32), a substituição apagava nome, foto e preço de manutenção, porque esses campos nunca tinham sido salvos nesse formato mais antigo — e não existiam no que estava sendo copiado por cima.

Confirmado por reprodução exata antes de mexer: simulei um save no formato antigo e vi o mesmo resultado do print do usuário (`undefined` no nome, sem foto, saúde correta).

Corrigido: a restauração agora guarda uma cópia do modelo original das 5 máquinas de fábrica ANTES de mexer em qualquer coisa, e usa esse modelo como base — o que o save realmente tinha (saúde, status, apelido,

etc.) entra por cima, o que faltava (nome, foto, preço) continua vindo do modelo original. Isso resolve o save antigo sem quebrar o save novo. Para máquina comprada vinda de um save no formato antigo (sem nome salvo — não tem como reconstruir com segurança), a entrada quebrada não é restaurada, em vez de aparecer como "undefined" na tela — é perda pontual de uma máquina antiga específica, não um bug novo se repetindo.

Validado: 11 verificações cobrindo os dois formatos de save lado a lado — antigo (nome/foto/preço voltam do modelo, saúde/apelido salvos aplicados por cima) e novo (máquina comprada continua restaurando completa, sem regressão). Regressão geral: 7 (máquina comprada) + 44 (campanhas) + 13 (fumaça completa) = 75/75.

Lição registrada: ao mudar o formato de dado salvo, sempre testar com um save do formato ANTERIOR também, não só com um save gerado pelo próprio código novo. A sessão 34 testou só o caminho "salvar com o código novo, restaurar com o código novo" — nunca testou "restaurar um save salvo pelo código velho", que é exatamente o que um jogador de verdade vive numa atualização.

Importante pro usuário: essa correção impede o problema de se repetir daqui pra frente, mas **não desfaz automaticamente** o que já aconteceu no jogo salvo atual — se o jogo já salvou de novo depois de mostrar "undefined" (o que é provável, já que salva sozinho a cada

30s e ao trocar de tela), esse estado quebrado pode já estar gravado. Melhor caminho: instalar esta versão corrigida e, se as máquinas ainda aparecerem erradas, começar uma conta nova — a partir de agora, esse tipo de perda não deve mais acontecer em atualizações futuras.

sw.js atualizado pra v3 (subiu de v2, seguindo a própria regra documentada na sessão 35 — toda publicação nova precisa subir esse número).

Arquivo gerado: `teste-migracao-save.js`.

Próximo passo real: subir esta versão pro GitHub. Nenhuma pendência de código nova em aberto além das já registradas (extraDias da opção de engenharia, balanceamento da Loja).

Sessão 35 — 06/09/2026 — Problema real de produção: atualização não pode depender de limpar cache manualmente

Pedido: o usuário levantou uma preocupação de arquitetura, não um bug pontual — com o jogo rodando pra várias pessoas, ninguém pode ter que limpar cache toda vez que sair uma atualização nova. Isso conecta direto com o diagnóstico da sessão 34 (o "jogo sem interação" provavelmente era cache de navegador não limpo).

**Causa raiz real, achada no próprio service worker
construído na sessão 31:**

1. A estratégia "rede primeiro" chamava `fetch(event.request)` sem nenhuma opção — isso ainda deixa o navegador aplicar o cache HTTP comum (via `Cache-Control` que o GitHub Pages manda), mesmo dentro do service worker. "Rede primeiro" só funciona de verdade se a busca ignorar esse cache também, não só o cache próprio do service worker.
2. Mesmo com o service worker pegando a versão nova, a aba já aberta continuaria rodando o JavaScript antigo em memória até alguém fechar e reabrir manualmente — nada recarregava sozinho.
3. O navegador só verifica se existe um `sw.js` novo de vez em quando por padrão (não a cada visita) — sem pedir a verificação de propósito, a atualização podia demorar a aparecer mesmo tecnicamente disponível.

Corrigido:

- `fetch(event.request, { cache: 'no-store' })` no service worker — ignora cache HTTP de verdade, sempre busca fresco quando online
- `CACHE_NAME` subiu de v1 pra v2 (documentado: precisa subir a cada publicação futura — é isso que faz o service worker antigo perceber a troca)
- No registro do service worker: chama `reg.update()` assim que registra, e de novo toda vez que o app volta a ficar visível (`visibilitychange`) — não

espera a checagem automática e esporádica do navegador

- Quando um service worker novo assume o controle da página (`controllerchange`), a página recarrega sozinha — com trava contra loop de recarregamento infinito

Validado com navegador real, não só leitura de código — usei Playwright com um Chromium já disponível no ambiente (a tentativa de baixar via rede falhou por restrição de acesso, mas havia uma instalação pronta em `/opt/pw-browsers`):

- Servidor HTTP de teste construído do zero, mandando `Cache-Control: max-age=600` de propósito (simulando o tipo de cache que o GitHub Pages manda) — pra provar que a correção ignora isso mesmo, não só em teoria
- Carreguei uma "versão 1" marcada, confirmei o service worker registrado e controlando a página
- Troquei o servidor pra "versão 2" (a versão real atual) **sem reiniciar o navegador, sem limpar nada manualmente**
- Simulei o app voltando ao primeiro plano — a página detectou a atualização sozinha e **recarregou por conta própria antes mesmo do próximo passo do teste rodar** (bom sinal, não erro — teve que ajustar o script de teste pra lidar com a rapidez disso)
- Título final confirmou a versão nova carregada — 3/3, prova real de que o mecanismo funciona de ponta a

ponta, não é só teoria de como service worker
"deveria" funcionar

**Regressão da lógica do jogo (jsdom, não afetada por
essa mudança que é só de infraestrutura de
atualização):** 44 (campanhas) + 13 (fumaça completa) +
7 (máquina comprada, sessão 34) = 64/64, nada
quebrou.

Nota de processo importante: esta foi a primeira vez
neste projeto que uma correção de service worker foi
validada com navegador real em vez de só leitura de
código ou teste jsdom (que não suporta service worker).
Vale manter esse padrão pra qualquer mudança futura
nessa área — o ciclo de vida de service worker tem
armadilhas (como a do cache HTTP dentro do próprio
SW) que só aparecem testando de verdade.

Arquivos gerados: `servidor-cache-teste.js`, `teste-
atualizacao-sw.js` — ambos reutilizáveis pra validar
qualquer mudança futura no service worker.

Próximo passo real: subir a versão corrigida pro GitHub.
Lembrete pra próxima publicação: subir `CACHE_NAME` no
`sw.js` de novo (v2 → v3, etc.) a cada atualização futura.

**Sessão 34 — 06/09/2026 — Bug real: máquina
comprada não sobrevivia à restauração +
diagnóstico de cache de navegador**

Pedido: o usuário reportou que, depois de instalar o jogo corrigido (sessão 32), o dinheiro e a sede restauravam certo, mas as 4 máquinas que ele tinha comprado sumiam — voltava sempre pras 5 iniciais. Relatou também "interação zero no Hub... cadê as imagens, os eventos" — o que descreveria um jogo sem praticamente nada construído desde a sessão 8.

Bug 1 — máquina comprada não restaurava (confirmado e corrigido):

- Achado por leitura: `restaurarGameState()` (sessão 32) tinha `if(!MACHINES[k]) return;` — só ATUALIZAVA máquina que já existisse na inicialização fresca da página (as 5 iniciais). Máquina comprada ganha uma chave nova, que nunca existe nesse momento — era descartada silenciosamente.
- Pior ainda: mesmo corrigindo esse filtro, `salvarGameState()` só guardava um subconjunto de campos (nem nome, foto ou preço de manutenção entravam) — não daria pra reconstruir a máquina comprada de forma completa.
- **Corrigido:** agora salva o objeto `MACHINES` inteiro (confirmado sem funções embutidas, seguro pra serializar), e a restauração limpa e repõe o objeto inteiro (`MACHINES` é `const`, então não dá pra reatribuir a variável — mas dá pra limpar e copiar as propriedades de volta no mesmo objeto).
- Validado com o cenário exato: comprar 4 máquinas de verdade pelo fluxo real (`buyMachine()`), salvar, simular reabertura completa da página com esse

save, confirmar 9 máquinas (não 5),
nomes/fotos/preços intactos. 7/7. Repetido no
arquivo já extraído do zip final, mesmo resultado.

**Bug 2 (investigado, não é bug de código) — "cadê as
imagens, os eventos":**

- Antes de suspeitar de cache, confirmei que o arquivo atual TEM todas as funcionalidades: painel de comando (5 ocorrências), eventos dinâmicos (4), mensagem do contratante (7), fotos de sede em base64 (5), quebra de máquina (5), Consultor (5) — nada foi perdido ou ficou de fora do arquivo real
- Perguntado o fluxo exato do usuário: ele desinstalou o PWA antigo, subiu tudo de novo pro GitHub, visitou o link, reinstalou, jogou
- **Diagnóstico:** desinstalar o atalho do PWA da tela inicial NÃO limpa o cache do navegador nem desregistra o service worker daquele endereço — isso fica guardado no Chrome por origem (domínio), independente do atalho existir ou não. Se o mesmo link do GitHub Pages já tinha sido visitado antes (em qualquer sessão anterior desta conversa, testando qualquer versão anterior), o navegador pode estar servindo arquivo antigo do cache HTTP normal ou do cache do service worker, mesmo com o app reinstalado do zero.
- **Não foi possível confirmar 100% sem acesso ao dispositivo do usuário** — fica registrado como diagnóstico mais provável, não certeza absoluta. Passo recomendado: limpar dados do site (não só

desinstalar o atalho) nas configurações do navegador antes de revisitar.

Regressão: 7 (máquina comprada) + 11 (restauração geral, sessão 32) + 44 (campanhas) + 13 (fumaça completa) = 75/75, nada quebrou.

Arquivo gerado: `teste-restore-maquina-comprada.js` .

Próximo passo real: aguardar confirmação do usuário sobre se limpar os dados do site resolveu o "jogo sem interação" — se não resolver, é preciso investigar mais fundo (possível problema na própria configuração do GitHub Pages, não no código).

Sessão 33 — 05/09/2026 — Primeiro playtest real (30 min) + travamento de contratos, missões recalibradas, e o Consultor

Pedido: o usuário jogou 30 minutos de verdade e trouxe 3 relatos: chegou fácil na 3ª sede (missões muito curtas), teve um momento em que quase todos os contratos disponíveis pediam máquina que ele não tinha (travou, só um contrato dava pra pegar), e apontou que o jogo "fica morto fora da obra" — sugerindo um consultor que desse conselhos reais, com exemplos concretos de fala.

Isso é o primeiro playtest formal de verdade desde a sessão 20 — o `GUIA-PLAYTEST.md` nunca foi preenchido like formulário, mas esse relato cobre praticamente as

mesmas perguntas centrais dele, com dado real de 30 minutos de jogo.

1. Travamento de contratos — confirmado, não presumido:

- Simulação confirmou: 37,5% do catálogo de 40 arquétipos pede máquina que a frota inicial não possui (achado original da sessão 22, nunca resolvido até agora)
- No sorteio específico gerado durante a investigação, só 3 dos 5 slots visíveis eram `hasMachine:true` — bate com o relato do usuário
- **Corrigido:** `regenerarContrato()` agora sorteia com peso — 80% das vezes vem de um arquétipo que a frota já cumpre, 20% do catálogo inteiro (mantém a aspiração de "preciso comprar tal máquina" sem deixar isso dominar os slots). Medido por simulação de 300 rodadas: média de slots usáveis subiu de ~3,1 para 4,68 de 5.

2. Missões recalibradas — curva inteira, não só remendo pontual:

- Campanha 2 (Garagem com Oficina): 3 → 6 contratos no prazo, 2 → 3 manutenções, R\$150k → R\$280k faturamento
- Campanha 3 (Sede Média): 1 → 2 contratos de alto risco, 3 → 5 usos de oficina, R\$500k → R\$900k faturamento, 8 → 15 contratos no prazo

- Campanhas 4 e 5 ajustadas proporcionalmente (~1,4-1,5x) pra manter a curva coerente, incluindo o limiar embutido em `_marcasConsolidadas()` (10 → 14)
- Testes de campanha (`teste6.js` , `teste-fumaca-final.js`) tinham os números antigos hardcoded — atualizados pros novos limiares, não é regressão real

3. O Consultor — Fase 3, item 11, agora com especificação real do usuário:

- Pessoa fixa e recorrente (Marisa Andrade, foto real), diferente dos contatos de cliente que trocam a cada contrato
- `gerarConselhoConsultor()` analisa o estado real do jogo em ordem de prioridade — nunca fala à toa: máquina quebrada (mais urgente) → sem contrato ativo com opção disponível → quase completando a próxima sede → caixa apertado → contrato pedindo máquina que falta (sugere vender uma ociosa e focar no tipo pedido, ou só sugere compra se não houver candidata a vender) → caixa saudável → nada urgente, fica quieto
- Moderação de verdade: só fala no Hub, cooldown de ~100s reais, e mesmo assim só 45% de chance por visita
- 2 bugs de texto encontrados e corrigidos durante o próprio teste: concordância de plural errada ("disponíveis") e um trecho que pegava só o primeiro

caractere de `missing` por tratá-lo como array quando na verdade é uma string única

Validado:

- 10 verificações do consultor (cada condição isolada, moderação/cooldown funcionando, só fala com Hub visível, foto/nome/cargo corretos)
- 2 verificações da nova distribuição de sorteio de contrato
- Regressão completa: 44 (campanhas, com números atualizados) + 12 (Hub vivo) + 13 (fumaça completa) = 69/69, mais os 80/80 já confirmados nas correções de pool — nada quebrou

Arquivos gerados: `teste-pool-pesado.js`, `teste-consultor.js`, `diag-pool-atual.js`.

Próximo passo real: sincronizar a versão final pro pacote GitHub e reempacotar. Depois, itens 10 (marcos da empresa) e 12 (diário de notícias mais vivo) da Fase 3 seguem pendentes — mas o consultor já cobre boa parte do "morto fora da obra" que motivava o item 12.

Sessão 32 — 05/09/2026 — BUG CRÍTICO:
restauração de jogo nunca funcionou de verdade

Pedido: usuário instalou o jogo, jogou, saiu, e ao voltar o progresso tinha zerado.

Investigação (do sintoma até a causa raiz):

1. Primeira hipótese: `acceptedContracts` (contratos em andamento) não estava sendo salvo em `salvarGameState()` — confirmado por leitura, era real, mas não explicava "zerou tudo", só explicaria perder o contrato específico em andamento.
2. Segunda hipótese: `beforeunload` não dispara de forma confiável quando um PWA instalado é fechado no celular (o sistema pode matar o processo sem esse evento) — real e relevante, mas ainda não explicava um "zero" completo.
3. **Causa raiz de verdade, achada testando o carregamento real da página (não manipulando estado direto via código, como todos os testes anteriores faziam):** `checkExistingAccount()` — a função que decide se mostra tela de login (jogador antigo) ou cadastro (novo) e que chama `restaurarGameState()` — era invocada na linha do arquivo **logo depois de ser definida**, mas `playerCash` (e outras variáveis de estado) só eram declaradas com `let` **centenas de linhas depois**. Isso é erro de zona morta temporal em JavaScript: toda vez que um jogador com conta e save existentes abria o jogo, `restaurarGameState()` lançava `ReferenceError: Cannot access 'playerCash' before initialization` — só que esse erro caía dentro do `try/catch` da própria função, que apenas registrava no console e retornava `false` **silenciosamente**. O jogo seguia carregando normalmente, só que com todas as variáveis nos valores padrão de um jogo novo. **Isso significa que a**

restauração de save nunca funcionou de verdade, em nenhuma das 31 sessões anteriores — só não tinha sido descoberto porque nenhum teste automatizado simulava o carregamento real de um jogador retornando (todos manipulavam estado depois da página já carregada, pulando exatamente o caminho de código que tinha o problema).

Corrigido:

- `checkExistingAccount()` agora é chamada só no final do script, depois que toda variável de estado já foi declarada — eliminando o erro de zona morta temporal de vez
- `salvarGameState()` / `restaurarGameState()` ampliadas pra cobrir o que também estava faltando: `acceptedContracts`, `completedContracts`, `lostContracts`, `financiamentosAtivos`, `historico`, `historicoCaixa`, `scheduledMaintenances`, o estado atual do catálogo de contratos (`CONTRACTS`), e a identidade de máquina da Fase 3 (apelido, dias de empresa, contratos realizados, faturamento gerado, manutenções feitas, estado de quebrada) — nenhuma dessas era salva antes
- Reforço de confiabilidade pra PWA em mobile: além do `beforeunload` (que já existia) e do auto-save a cada 30s, agora também salva em `visibilitychange` (quando a aba/app fica escondida) e `pagehide` — muito mais confiáveis que

`beforeunload` quando o sistema operacional do celular suspende ou mata o app

Validado com o cenário exato do problema relatado — carregamento real da página com conta e save pré-existent no localStorage, não manipulação de estado pós-carregamento:

- 11 verificações: nenhum erro de zona morta temporal, caixa/sede/reputação/faturamento restaurados com o valor salvo (não o padrão), **o contrato em andamento sobrevive** (a perda mais grave do problema original), histórico e financiamento preservados, tela de login mostra "jogo restaurado"
- 3 verificações confirmando que jogador **novor** (sem save nenhum) continua indo pra tela de cadastro normalmente — a correção não quebrou esse caminho
- Regressão completa no arquivo corrigido: 44 (campanhas) + 13 (fumaça completa) + 10 (quebra de máquina) + 15 (identidade de máquina) + 5 (mensagem de risco) = 87/87, nada quebrou

Lição registrada para não se repetir: testes que manipulam estado diretamente via `w.eval()` depois da página já ter carregado **não testam o caminho real de carregamento inicial**. Qualquer bug que more especificamente na sequência de inicialização (ordem de declaração, `checkExistingAccount`, primeira renderização) fica invisível pra esse estilo de teste. Daqui

pra frente, mudanças que tocam nessa área precisam de teste com `beforeParse()` populando `localStorage` **antes** do HTML carregar, replicando o cenário real de um jogador retornando — exatamente como foi feito nesta sessão.

Arquivo corrigido: `01-JOGO/app.html`, `_PACOTE-GITHUB/index.html`, `_PACOTE-GITHUB/app.html`, `github-package.zip` (todos ressincronizados e re-testados).

Arquivos de teste gerados: `teste-restore-fix.js`, `teste-jogador-novo.js`, `teste-crash-restore.js` (diagnóstico).

Próximo passo real: subir a versão corrigida pro GitHub (a que já estava lá, se chegou a subir, tinha esse bug). Depois, testar de novo no celular — instalar, jogar um pouco, fechar de verdade (não só minimizar), reabrir, confirmar que o progresso persiste.

Sessão 31 — 04/09/2026 — Pacote para GitHub + 3 bugs de PWA encontrados e corrigidos

Pedido: montar o pacote pronto pra subir no repositório GitHub.

Achados antes de empacotar (não presumidos — confirmados por leitura direta):

1. `manifest.json` referenciava `icon-192.png` e `icon-512.png` (sem `"-maskable"`) — **esses arquivos**

nunca existiram na pasta, só as versões maskable. O manifest apontava pra arquivo inexistente.

2. `app.html` **não tinha nenhum link pro manifest.json**, nem meta `theme-color`, nem registro de service worker — zero busca encontrou essas strings no arquivo inteiro. A "instalação como PWA" nunca funcionou de verdade, apesar do manifest existir.
3. **Não existia** `sw.js` (service worker) em lugar nenhum do projeto — sem ele, não há cache offline, só o próprio navegador dependendo de cache HTTP padrão.
4. **Inconsistência dentro do próprio** `REGISTRO.md` : a seção 1 tinha uma frase antiga ("Fase 2 do roadmap continua pausada aguardando o playtest") que sobrou de uma edição da sessão 21 e nunca foi atualizada nas 9 sessões seguintes — contradizia o cabeçalho da mesma seção, que já dizia corretamente "Fase 2 completa". Corrigido antes de escrever o README em cima do arquivo, pra não propagar a inconsistência.

Corrigido:

- `manifest.json` : os 2 ícones que existem de verdade (maskable) agora cobrem os dois propósitos (`"any maskable"`), sem referência a arquivo inexistente
- `app.html / index.html` : adicionado `<link rel="manifest">`, `theme-color`, `apple-touch-icon`, e registro de service worker no `window.addEventListener('load', ...)` já

existente (com fallback silencioso se falhar — não é bloqueante)

- Criado `sw.js` — estratégia network-first com fallback pro cache, cacheando só os arquivos essenciais (o jogo inteiro já vem embutido em base64 dentro do HTML, então cachear 1 arquivo já dá acesso offline completo)
- Corrigida a inconsistência do `REGISTRO.md` (parágrafo stale reescrito pra refletir o estado real pós-sessão 30)

Decisão estrutural importante: os arquivos internos do projeto sempre viveram em `01-JOGO/`, `02-DADOS/`, `03-PLANO/` (convenção de 30 sessões, não alterada agora). Mas pra o GitHub Pages servir o jogo na raiz do site, `index.html` precisa estar na **raiz do repositório**, não dentro de subpasta. Resolvido criando um pacote **separado** (`_PACOTE-GITHUB/` + `github-package.zip`) com a estrutura correta pra upload direto — sem mexer na organização interna de desenvolvimento, que continua em `01-JOGO/` como sempre foi.

Criado: `README.md` na raiz do pacote — descrição do jogo, o que já existe (resumido, com link pro `REGISTRO.md` pra detalhe completo), estrutura de pastas explicada, nota clara de que as fotos/vídeos soltos são só referência (o jogo não lê mais deles), e instrução de como rodar os testes.

Validado:

- Sintaxe do `index.html` empacotado, verificada isoladamente
- Teste de fumaça de ponta a ponta (13 verificações) rodado **direto no arquivo que vai pro GitHub**, não numa cópia — login, navegação pelas 8 telas, aceite de contrato, compra de sede, salvar o jogo
- Confirmado que a correção do `REGISTRO.md` foi incluída no pacote antes de zipar

Arquivos gerados: `_PACOTE-GITHUB/` (pasta com a estrutura pronta pra upload), `github-package.zip` (mesmo conteúdo compactado, 2,5 MB), `README.md` .

Próximo passo real: subir o conteúdo de `_PACOTE-GITHUB/` (ou extrair o zip) pra raiz do repositório `wanderpomares-sys/ImperioDasMaquinas` , ativar GitHub Pages apontando pra branch/pasta raiz, e testar a instalação como PWA de verdade num celular (algo que nunca foi possível confirmar antes, já que a ligação `manifest` → HTML nunca existiu até esta sessão).

Sessão 30 — 04/09/2026 — Fase 3, item 9: identidade das máquinas

Pedido: "Continua" — usuário pediu resumo da Fase 3 (identidade das máquinas, marcos da empresa, mentor, diário mais vivo) e decidiu seguir com ela.

Feito:

- Cada máquina ganha um **apelido** sorteado de um pool de 10 (Bruta, Coração de Aço, Fura-Fila, etc.) – atribuído na criação, seja frota inicial ou compra na Loja
- **4 estatísticas de histórico reais**, incrementadas nos pontos certos do código, não decorativas:
 - `diasNaEmpresa` – conta todo dia, trabalhando ou parada (diferente de `idadeDias`, que é desgaste mecânico)
 - `contratosRealizados` – soma 1 pra cada máquina que participou de um contrato concluído com sucesso (nunca em contrato perdido/cancelado)
 - `faturamentoGerado` – soma o valor do contrato pra cada máquina que participou
 - `manutencoesFeitas` – conta nos 3 caminhos de manutenção que existem (agendada normal, oficina parceira, reparo de emergência da sessão 29)
- Exibido no detalhe de Manutenção: apelido no título, e uma caixa "📖 História da [apelido]" com as 4 estatísticas
- **Selo de veterana** (🏆) quando a máquina passa de 15 contratos realizados

Validado:

- 15 verificações de inicialização, contabilização nos 3 caminhos de manutenção, exibição visual, e o selo de

veterana aparecendo/sumindo no limiar certo (14 vs 15 contratos)

- 2 verificações de que `contratosRealizados / faturamentoGerado` sobem certo na conclusão real de um contrato

Achado durante o teste, não é bug — é efeito colateral

bom de correções anteriores: o teste de quebra de máquina da sessão 29 (que mockava 200 tentativas num único contrato) começou a falhar. Investigado: não é regressão. A fórmula de prazo recalibrada nas sessões 22-24 faz o contrato concluir bem mais rápido e previsível agora (~10-15 dias reais), então um único contrato não vive tempo suficiente pra dar 200 chances de verdade pra um evento de 7%/dia — a conta antiga de "200 tentativas garantem quase certeza" não vale mais, porque o teto real de tentativas é o tempo de vida do contrato, não um número arbitrário. Corrigido o teste pra repetir com um contrato novo sempre que o atual concluir sem a máquina quebrar (até 12 ciclos) — o mecanismo de quebra em si nunca parou de funcionar, só ficou mais raro de disparar dentro de um contrato só, porque os contratos agora são mais curtos e previsíveis. Substituí o script de teste antigo por essa versão mais robusta.

- Regressão: 44 (campanhas) + 10 (quebra de máquina, reconfirmada) + 12 (decisão de antecipação) + 11 (personalidade de contrato) = 77/77, nada quebrou de verdade

Arquivos gerados: `teste-identidade-maquina.js`, `teste-historico-maquina-contrato.js`, `teste-quebra-maquina.js` (substituiu a versão da sessão 29, mais robusta a contratos curtos).

Próximo passo real: itens 10 (marcos e conquistas da empresa), 11 (mentor) e 12 (diário de notícias mais vivo) da Fase 3.

Sessão 29 — 04/09/2026 — Fase 2, item 8: quebra de máquina dramática (Fase 2 completa)

Pedido: continuar com consequências visíveis de máquina quebrada, mas com peso visual de verdade — nada de mensagem simples de texto. O usuário foi enfático: "uma máquina quebrada é terrível, dor de cabeça e prejuízo certo", e isso devia se refletir na apresentação.

Achado antes de construir: zero menções a "quebra"/"avaria"/"pane" existiam no código inteiro. Confirmado: hoje era só a saúde caindo silenciosamente, exatamente o "número na ficha" que o roadmap aponta como insuficiente.

Construído do zero:

- **Mecânica real de quebra:** máquina em contrato (`inContract`), com saúde abaixo de 35%, tem 7% de chance por dia de quebrar de verdade — separado do evento de risco e do evento de sabor, com prioridade

sobre este último (quebra nunca compete com evento de sabor no mesmo dia)

- **Custo imediato:** R\$3.000 a R\$8.000 de custo emergencial (reboque + diagnóstico), debitado na hora
- **Consequência real na produtividade:** máquina quebrada é excluída do cálculo de `calcularFatorProdutividade` — não conta mais nem pra saúde média nem pra quantidade disponível. Testado isoladamente: 2 máquinas saudáveis davam fator 0,96; quebrando 1 das 2, cai pra 0,48 — a "capacidade de operar" cai de verdade, não é só cosmético
- **Apresentação dramática (o que foi pedido explicitamente):** tela cheia vermelho-escura, ícone tremendo (`shake` , 4 ciclos), destroços caindo (    , 20 partículas, rotação e queda), flash vermelho na entrada, foto da máquina dessaturada, texto do que quebrou (5 variações rotativas), custo emergencial em destaque, botão que leva direto pra manutenção daquela máquina
- **Reparo de emergência:** único jeito de tirar a máquina do estado quebrado — custa 1,8× o preço da peça original (bem mais que manutenção agendada normal, de propósito, porque é emergência não planejada), restaura a saúde pra pelo menos 55%
- **Badges visuais em 3 telas:** Manutenção, Máquinas e Hub (nesse último, máquina quebrada sempre

aparece primeiro na fileira — é a coisa mais urgente que existe na frota)

Validado:

- 17 verificações do fluxo completo: quebra dispara, custo debitado, overlay dramático abre com foto/nome/destroços certos, histórico e notícia registrados, fechar overlay navega pra manutenção da máquina certa, bloco de reparo aparece, reparar tira do estado quebrado e cobra o valor certo
- 4 verificações dos badges nas 3 listas, incluindo confirmar que o badge some depois do reparo
- 2 verificações isoladas do impacto real na produtividade (0,96 → 0,48 ao quebrar 1 de 2 máquinas)
- Um teste inicial falhou completamente (17 falhas) por eu ter mockado `Math.random()` num valor baixo demais — isso também disparou o "pedido de antecipação" (checado ANTES da quebra na mesma função), retornando cedo antes de qualquer código novo rodar. Não era bug do jogo. Corrigido com sorteio real em loop, resolvendo risco intercorrente quando aparecia no meio do caminho, do mesmo jeito que já tinha resolvido esse exato problema de teste antes nesta sessão (personalidade de contrato, sessão 28)
- Regressão: 44 (campanhas) + 12 (decisão de antecipação) + 11 (personalidade de contrato) + 6 (bug do atraso invisível) = 73/73, nada quebrou

FASE 2 DO ROADMAP-GAMEFEEL.MD ESTÁ COMPLETA:

decisões durante o contrato (item 5, sessão 27), eventos derivados do estado real (item 6, já atendido desde a sessão 15), contratos com personalidade (item 7, sessão 28), consequências visíveis de máquina quebrada (item 8, sessão 29).

Arquivos gerados: `teste-quebra-maquina.js`, `teste-badges-quebra.js`, `teste-fator-productividade-quebra.js`.

Próximo passo real: decisão entre Fase 3 (apego — identidade das máquinas, marcos da empresa, mentor) ou pausar de novo pra playtest, já que duas fases inteiras (1 e 2) foram implementadas desde a última vez que um humano jogou e reportou de verdade (sessão 21 foi a última rodada de feedback real antes dessas 8 sessões de feature nova).

Sessão 28 — 04/09/2026 — Fase 2, item 7: contratos com personalidade

Pedido: "Ok prossiga" — seguir com o item 7 do roadmap, na ordem indicada na sessão anterior.

Feito:

- Sorteio de personalidade a cada regeneração de contrato: 12% tentador, 15% especial, 73% comum
- **Tentador:** sobrescreve valor ($\times 1,6$) e risco (força "Muito Alto") do arquétipo sorteado, com texto de

expectativa rotativo ("Essa obra pode mudar sua empresa.", etc.) exibido em destaque no detalhe do contrato. Badge 🔥 na lista.

- **Especial:** mantém valor/risco do arquétipo normal, mas rende +8 de reputação extra na conclusão — **só se entregue no prazo** (a personalidade não isenta de cumprir o combinado, só recompensa mais quando cumpre). Badge ★ na lista, aviso no detalhe.
- **Comum:** sem mudança nenhuma — é a maioria dos contratos, como já era.

Bug real encontrado e corrigido no caminho, não

relacionado à minha implementação nova: a tabela de penalidade de atraso/cancelamento usava

`arquetipo.risk` (o risco ORIGINAL do sorteio) em vez de `c.risk` (o risco FINAL do contrato, depois de qualquer sobrescrita). Isso significa que, mesmo antes de existir personalidade de contrato, qualquer futuro ajuste de risco pós-sorteio nunca teria refletido na multa de verdade — hoje ficou visível porque o "tentador" força risco pra "Muito Alto" e eu conferi se a multa acompanhava. Corrigido para usar `c.risk`.

Validado:

- 11 verificações: distribuição do sorteio bate com as porcentagens configuradas (testado com 3.000 amostras), tentador sobrescreve valor/risco/texto corretamente, a multa usa o risco final (não o original), badges aparecem certos na lista e no detalhe pra tentador e especial

- 3 verificações do bônus de reputação: contrato especial concluído no prazo rende +10 no total (2 normal + 8 do bônus), notícia registrada
- Um teste inicial travou (não terminou dentro do tempo) por usar `Math.random()` mockado num valor constante — isso pode causar loop infinito no sorteio de arquétipo do próprio `regenerarContrato` (que sorteia até achar um nome diferente do atual, e com valor de sorteio fixo pode nunca encontrar). Corrigido usando sorteio real repetido até sair o resultado desejado, em vez de mockar o gerador de números aleatórios
- Regressão: 44 (campanhas) + 12 (decisão de antecipação) + 6 (bug do atraso invisível) = 62/62, nada quebrou

Arquivos gerados: `teste-personalidade-contrato.js`, `teste-bonus-contrato-especial.js`.

Próximo passo real: item 8 da Fase 2 (consequências visíveis — máquina quebrada de verdade reduz capacidade de operar, não só um número na ficha), ou item 6 (revisão dos eventos derivados do estado real, já parcialmente atendido desde a sessão 15).

Sessão 27 — 03/09/2026 — Fase 2, item 5: decisão de antecipação do cliente

Pedido: "Próxima fase" → "Siga o roadmap" — começar a Fase 2 do `ROADMAP-GAMEFEEL.md` ("Criar tensão"), item 5:

decisões reais durante o contrato.

Feito:

- Reaproveitada a infraestrutura de mensagem do contratante (sessões 16-17): o modal ganhou um segundo modo, `abrirDecisaoCliente()`, que troca o botão único "Entendido" por 2-3 botões de escolha real, mantendo a mesma cara visual (foto, nome, cargo, balão de mensagem)
- Novo evento: **pedido de antecipação**. Durante o contrato (fora de situação de risco), 10% de chance por dia de o cliente pedir pra terminar 2 dias antes — só uma vez por contrato, só em contratos com prazo ≥ 8 dias, só na janela de 5% a 70% de progresso (não faz sentido pedir isso no início nem quase no fim)
- **3 saídas reais, com trade-off de verdade:**
 1.  Aceitar — prazo cai 2 dias, +5 reputação, risco diário sobe 40% até o fim do contrato
 2.  Recusar — nada muda, opção segura
 3.  Reforçar com máquina extra — só aparece se existir de verdade uma máquina livre do mesmo tipo já usado no contrato; custa 8% do valor do contrato, adiciona a máquina ao contrato (sobe `fatorQuantidade` de verdade), reduz prazo em 2 dias, **sem** aumentar risco
- A decisão consome o dia (como já acontece com situação de risco) — o jogador precisa avançar de novo depois de escolher

Validado:

- 12 verificações do fluxo principal (gatilho força, decisão abre, botão "Entendido" some, escolher "aceitar" aplica os 3 efeitos certos, não repete no mesmo contrato)
- 1 verificação confirmando que, com a frota inicial (1 de cada tipo), a opção de reforço corretamente NÃO aparece
- 7 verificações forçando um cenário com máquina extra disponível, confirmando que a opção de reforço aparece e cada efeito é real (dinheiro debitado, máquina entra no contrato, prazo cai, risco NÃO sobe — diferente da opção de aceitar)
- Regressão: 44 (campanhas) + 6 (bug do atraso invisível) + 5 (mensagem de risco) + 5 (mensagem de conclusão) = 60/60 — importante ter checado, já que a mudança tocou a mesma infraestrutura de modal usada pelas 3 mensagens de contratante já existentes

Arquivos gerados: teste-decisao-antecipacao.js , teste-reforco-sem-maquina.js , teste-reforco-com-maquina.js .

Próximo passo real: os itens 6, 7 e 8 da Fase 2 — eventos derivados do estado real (6, já parcialmente atendido desde a sessão 15 via EVENTOS_CONTRATO + calcularExposicao , mas vale revisão), contratos com personalidade (7), consequências visíveis de máquina quebrada (8).

Sessão 26 — 03/09/2026 — Correção real: animação do painel só sincroniza quando o Hub está visível

Pedido: o usuário testou a animação da sessão 25 e não viu nada mudar. Pedi pra só explicar antes de mexer — expliquei a causa (o painel é re-renderizado em segundo plano toda vez que algo acontece no jogo, mesmo com o Hub fora de tela, então o "valor antigo" já estava sincronizado com o atual antes do jogador chegar lá pra ver). O usuário confirmou pra seguir com a correção.

Causa raiz confirmada: `renderHubExtras()` é chamado de pelo menos 3 lugares que nada têm a ver com o jogador estar olhando pro Hub — `registrarHistorico()` (disparado o tempo todo), `ajustarReputacao()`, e `resgatarMissao()`. Cada uma dessas chamadas atualizava `ultimoCaixaPainelComando / ultimaReputacaoPainelComando` pro valor atual, mesmo com a tela do Hub escondida — consumindo a "distância" que a animação precisaria pra ter algo visível pra percorrer.

Corrigido: o rastreamento (e o disparo da animação) agora só acontece quando `screen-hub` tem a classe `active` de verdade. Enquanto o jogador está em outra tela, o "último valor mostrado" fica congelado — só é atualizado (e animado) na próxima vez que o Hub realmente aparecer na tela. Confirmado que `goTo()` aplica a classe `active` antes de chamar os renders,

então a checagem funciona no momento exato da navegação.

Validado com o cenário real de jogo, não só chamada direta de função:

- Visitar o Hub pela primeira vez (mostra direto, sem de onde animar)
- Sair do Hub, mudar o caixa em outra tela (dispara `renderHubExtras()` em segundo plano) – confirmado que o rastreamento NÃO se move enquanto o Hub está escondido
- Voltar ao Hub – a animação agora dispara de verdade, visível, chegando no valor exato depois de ~700ms
- Repetir o ciclo (sair, mudar, voltar) uma segunda vez – confirma que não "gasta" o efeito só na primeira vez
- Mudança de valor com o Hub já visível continua animando normalmente (não quebrou o caso que já funcionava)
- 8/8 nesse teste; regressão completa $8+7+12+44 = 71/71$, nada quebrou

Nota para o registro: esta é a segunda vez nesta sessão de trabalho que uma "correção" validada por teste automatizado não correspondia à experiência real do jogador – a primeira (sessão 25) validou o mecanismo isoladamente sem reproduzir o padrão real de uso (sair da tela, fazer outra coisa, voltar). Fica como lembrete: daqui pra frente, qualquer teste de UI que dependa de "o

jogador estava olhando" precisa simular navegação real (`goTo()` pra fora e de volta), não só chamar a função de render diretamente.

Arquivo gerado: `teste-animacao-painel-cenario-real.js`.

Sessão 25 — 03/09/2026 — Microanimação do painel de comando do Hub (pendência da sessão 21 fechada)

Pedido: "E as micro animações no Hub?" — o usuário lembrou da pendência pequena registrada desde a sessão 21: os cards de Caixa e Reputação do painel de comando não animavam como o resto do jogo (topbar, tela de Finanças).

Causa confirmada: `renderPainelComando()` reconstrói os 5 cards inteiros via `innerHTML` a cada chamada — isso destrói e recria os elementos DOM toda vez, então `animarNumero()` (que precisa segurar a mesma referência de elemento durante ~550ms de transição) nunca tinha chance de funcionar ali, mesmo já existindo e funcionando em outros lugares do jogo desde antes.

Corrigido:

- IDs estáveis (`painelCaixaValor` , `painelRepValor`) nos dois números
- Duas variáveis novas de rastreamento (`ultimoCaixaPainelComando` ,

`ultimaReputacaoPainelComando`) – separadas de `caixaUltimaVisitaHub`, que serve outro propósito (o texto "desde sua última visita")

- Depois do `innerHTML` montar o card, `animarNumero()` é chamado nos dois elementos, contando com o valor anterior guardado – a mesma função que já anima caixa/reputação em outros lugares, sem duplicar lógica nova

Validado:

- 7 verificações: primeiro render mostra o valor direto (sem "de onde animar"), segundo render entra em transição visível (não salta), depois de ~700ms chega no valor exato, funciona igual pra reputação, e um render repetido sem mudança de valor não fica re-disparando animação à toa
- Regressão: 12 (Hub vivo) + 44 (campanhas de sede) = 56/56, nada quebrou

Arquivo gerado: `teste-animacao-painel-comando.js`.

Status da Fase 1 do Roadmap de Game Feel: com isso, não sobra nenhuma pendência conhecida de microanimação. Único item de polimento visual restante do roadmap inteiro seria itens da Fase 4 (som, transições entre telas), que nunca entraram em escopo.

Sessão 24 – 03/09/2026 – Correção definitiva:
atraso sem incidente eliminado

Pedido: o usuário reportou "de novo" — outro contrato com elogio do cliente, ótimo desempenho, nenhum problema, ainda assim entregue com atraso. Foi direto e categórico: "se não teve problemas, cliente elogiou, teve bom desempenho, então tem que ser obra entreguei no prazo. Simples."

Isso reformulou o diagnóstico. As sessões 21-23 tinham corrigido um bug de classificação e ajustado a média/variação da fórmula, mas ainda tratavam o atraso como algo parcialmente probabilístico mesmo sem incidente. O usuário apontou a lógica certa: **ausência de risco deveria significar praticamente sempre no prazo** — atraso devia vir de gestão de risco (a filosofia central do jogo: "Risco não é sorte. É gestão."), não do sorteio diário comum.

Medido antes de mexer: rodei o fluxo real (seguro completo, pra minimizar risco e isolar o efeito do sorteio) e contei separadamente os contratos que NUNCA tiveram um evento de risco. Resultado com a fórmula da sessão 23 (média 1,005): **67% desses contratos sem nenhum problema ainda terminavam atrasados** — confirmando exatamente a reclamação do usuário, com número exato, não "talvez".

Corrigido: testei uma sequência de médias diretamente no fluxo real (não só matemática pura) até achar o ponto de virada: com média em torno de 1,06, contratos sem incidente já batem 100% no prazo. Apliquei 1,08 (min 1,04 + spread 0,08) como margem confortável acima desse limiar.

Validado com o fluxo real, separando por causa:

- **Sem nenhum evento de risco: 54 no prazo, 0 atrasado — 100%**
- **Com evento de risco (opção mais cara escolhida): 0 no prazo, 15 atrasado — 100% atrasado**

A divisão agora é exatamente causal, como o usuário descreveu: atraso só acontece quando há risco de verdade.

Achado que fica ainda mais evidente e urgente com essa correção: como a margem "sem incidente" ficou justa (finaliza bem perto do próprio prazo, sem sobra), **qualquer evento de risco — mesmo resolvido com a opção mais cara — hoje garante atraso 100% das vezes**, porque a opção mais cara só reduz a chance de perder o contrato, não o custo de `extraDias`. Essa é a mesma pergunta em aberto desde a sessão 23, agora mais visível: **pagar mais pela solução deveria comprar também menos (ou nenhum) impacto no cronograma, não só menos risco de perder o contrato?**

Regressão completa: 44 (campanhas) + 6 (bug do atraso invisível, sessão 21) + 12 (Hub vivo) = 62/62, nada quebrado.

Arquivos gerados: `diag-sem-risco.js`, `ajusta-media-para-sem-risco.js`, `valida-formula-final-risco-vs-sem-risco.js`.

Próximo passo real: decisão do usuário sobre `extraDias` na opção mais cara de engenharia —

reduzir/zerar o impacto no prazo pra quem paga mais, ou manter como está (atraso garantido sempre que há risco, independente da opção escolhida)?

Sessão 23 – 03/09/2026 – Fórmula de prazo implementada + descoberta: risco custa tempo mesmo na opção mais cara

Pedido: o usuário escolheu, entre as 4 opções levantadas na sessão 22, "menos variação aleatória (mais controle, menos sorte)".

Achado 1 – reduzir variação sozinha não muda a taxa de sucesso: testei antes de implementar. Com a média do multiplicador fixa em 1.0, estreitar a faixa de [0,85–1,15] pra qualquer coisa mais estreita manteve a taxa de sucesso em ~50%, sempre. Isso é matemática, não escolha de calibração: quando a média é exatamente 1.0, a distribuição de resultados fica simétrica em torno do prazo – metade antes, metade depois – independente da variância. Reduzir variação sozinha só deixa os resultados mais **previsíveis** (menos extremos), não mais **prováveis de dar certo**. Expliquei isso ao usuário antes de prosseguir, já que "menos sorte" sozinho não resolveria a queixa original.

Achado 2 – o código já tinha uma tentativa de correção, com um erro de aritmética: ao abrir `avancarContrato` pra aplicar a mudança, encontrei que a fórmula **já tinha sido alterada** numa parte anterior desta mesma sessão

(não visível no momento) para `0,98 + Math.random()*0,08`, com um comentário dizendo que isso daria "média 1,005... ~75% no prazo". Não aceitei o comentário por leitura — simulei a fórmula exata do arquivo e o resultado real era **99,8% no prazo** (a média verdadeira de `0,98+0,08` é 1,02, não 1,005 — o valor mínimo estava digitado errado). Corrigido para `0,965 + Math.random()*0,08`, que simulação confirma dar **74,8%** — batendo com a meta que o próprio comentário já tinha estabelecido.

Achado 3 — o número real de jogo é bem mais baixo que a matemática pura, e a causa é outra: rodei 100 contratos pelo fluxo real (`acceptContract()` de verdade, saúde sempre 100%, resolvendo todo risco com a opção mais cara — replicando exatamente o que o usuário relatou ter feito). Resultado real: **37% no prazo, 58% atrasado, 5% perdido** — bem abaixo dos 74,8% da matemática pura.

- No caminho, encontrei um bug no meu **próprio script de teste**: as opções de engenharia estão ordenadas [mais cara/seguraprimeiro, intermediária, mais barata/arriscada por último], e eu tinha pego `opcoes[length-1]` (a mais barata) em vez de `opcoes[0]` (a mais cara) — o oposto do que o usuário descreveu fazer. Corrigido, e a contagem de resultado também tinha um bug (lia o último item de `completedContracts` mesmo quando o contrato tinha sido **perdido**, não concluído, inflando o número). Corrigido com comparação de tamanho de

array antes/depois de cada tentativa, não "pegar o último item".

- Causa real do 37%: `resolverComOpcaoEngenharia()` soma `opcao.extraDias` (1 a 2 dias, mesmo na opção mais cara) ao prazo decorrido **sem gerar nenhum progresso correspondente** — um custo de cronograma puro. Contratos de risco mais alto têm probabilidade diária de evento significativa o bastante pra que, ao longo de 10-20 dias, a chance de pelo menos 1 evento ocorrer seja frequentemente alta — e cada ocorrência custa dias de graça, mesmo pagando pela solução mais secura.

Implementado: a correção da fórmula diária (achado 2) — isso é uma correção de bug de aritmética que já tinha decisão do usuário por trás (sessão 22), não uma nova decisão de design. Testado e íntegro na regressão.

Não implementado, decisão pendente (achado 3): se resolver risco com a opção mais cara deveria custar menos tempo (ou nenhum) — hoje ela só reduz a chance de o contrato ser perdido, não o impacto no cronograma. Não mudei isso sozinho porque é uma escolha de design sobre o que "pagar mais" deveria comprar, não uma inconsistência lógica clara como a da sessão 21.

Validado:

- Sequência de simulações (faixas de variação, combinações média+variação, busca fina de média) documentando cada passo do raciocínio antes de decidir o número final

- Regressão completa: 44 (campanhas) + 6 (bug do atraso invisível, sessão 21, continua corrigido) + 12 (Hub vivo) + 15 (feedback proporcional) = 77/77, nada quebrou

Arquivos gerados: `testa-faixas-variacao.js`, `testa-combinado-media-variacao.js`, `testa-fino-media.js`, `verifica-formula-atual.js`, `acha-media-certa.js`, `simula-fluxo-real-com-risco.js`.

Próximo passo real: decisão do usuário sobre o achado 3 — a opção mais cara de engenharia deveria reduzir/eliminar o custo de tempo (`extraDias`), não só a chance de escalada? Sem essa decisão, o número real de sucesso no prazo continua bem abaixo da meta de ~75% que motivou o ajuste desta sessão.

Sessão 22 — 03/09/2026 — Achado: a fórmula de prazo é matematicamente apertada, não só o bug da sessão 21

Contexto: depois de corrigir o bug do "atraso invisível" (sessão 21), fui verificar se essa correção sozinha explicava o relato completo do usuário ("não consegui executar nenhum contrato dentro do prazo"). Não presumi que sim — testei.

Investigação:

1. Simulei 200 tentativas de contrato usando o array `acceptedContracts` diretamente (bypassando

`acceptContract()`) e a maioria falhou por falta de máquina — mas isso revelou que meu teste estava pulando a proteção real do jogo, não testando o jogo de verdade.

2. Descobri e confirmei que o jogo **já tem proteção against isso**: `acceptContract()` bloqueia com 
Você precisa da máquina exigida se `!c.hasMachine`, e `maquinasDisponiveisAgora()` bloqueia se a máquina certa já está ocupada em outro contrato. Um jogador real nunca consegue aceitar um contrato sem a frota mínima — isso não é a causa do problema relatado.
3. Refiz a simulação usando o fluxo real (`acceptContract()` de verdade, 100 contratos, saúde resetada a 100% antes de cada um, sempre a opção mais cara em situação de risco — replicando exatamente o que o usuário disse ter feito): **59 no prazo, 41 atrasados**, mesmo em condições ideais.
4. Isolei a fórmula de progresso pura (sem desgaste, sem risco, produtividade=100% fixa) em 2.000 simulações: tempo médio real pra concluir um contrato de prazo=10 dias é **10,5 dias**, e só **50,1%** terminam no prazo.

Causa raiz confirmada (matemática, não bug de código):
o progresso avança em incrementos aleatórios diários ($(100/\text{prazoDias}) \times [0,85 \text{ a } 1,15]$) e só "fecha" o contrato quando cruza 100% num dia inteiro — o excedente do dia em que cruza é descartado, nunca fica guardado pra compensar dias ruins anteriores. Isso cria

um viés estrutural: o tempo médio real de conclusão sempre passa um pouco do prazo nominal, mesmo com produtividade perfeita o tempo inteiro. **Terminar no prazo está desenhado pra ser parecido com cara ou coroa, mesmo jogando bem — não é falta de sorte do usuário, é a fórmula.**

Decisão tomada: não mudei a fórmula. Isso é uma escolha de dificuldade/balanceamento, não um bug — diferente da correção da sessão 21 (que era claramente uma inconsistência lógica). Registro o achado com os números exatos para o usuário decidir conscientemente se quer: (a) aumentar a média do multiplicador diário pra ficar acima de 1.0 (dar folga), (b) estreitar a variação aleatória (menos sorte/azar, mais controle), (c) aumentar levemente o prazo de cada arquétipo no `CONTRACT_POOL`, ou (d) manter como está — pode ser intencional que "no prazo" seja uma vitória de verdade, não o padrão esperado.

Nenhuma mudança de código nesta sessão — é achado de diagnóstico, aguardando decisão de design.

Arquivos gerados: `simula-taxa-prazo-fluxo-real.js`, `diag-formula-ritmo-matematico.js`, `check-pool-requisitos-frota.js`.

Achado secundário, também sem mudança de código: 8 dos 40 arquétipos de contrato (20%) pedem tipo de máquina (motoniveladora, escavadeira30) que a frota inicial nunca possui, e mais 7 (17,5%) pedem quantidade maior do que a frota inicial tem (ex: 2 caminhões, só 1 de

início) — 37,5% do catálogo. Isso **não causa bug** (o jogo bloqueia o aceite corretamente), mas significa que boa parte dos contratos sorteados nos 5 slots visíveis não são nem aceitáveis no início de jogo, reduzindo a sensação de escolha. Fica registrado, sem decisão tomada.

Próximo passo real: decisão do usuário sobre a fórmula de prazo (opções a-d acima). Sem essa decisão, não há mais código a escrever nessa frente.

Sessão 21 — 03/09/2026 — Bug do "atraso invisível" corrigido + indicador de ritmo vs. prazo

Pedido: o usuário reportou, jogando de verdade, que TODOS os contratos terminavam atrasados, inclusive um que não teve nenhum problema visível, com elogios do cliente durante a execução, e mesmo assim a mensagem final dizia "apesar do atraso, o resultado foi bom" — sem nenhum aviso de atraso ter aparecido antes. Também pediu um indicador de desempenho vs. prazo durante a obra, já que hoje só se descobre o atraso quando (e se) um alerta aparece.

Investigação (nunca aceitei a causa por suposição — testeí cada hipótese):

1. Simulei `fatorProdutividade` com a frota inicial real contra os 5 contratos ativos: confirmado que contratos exigindo tipo de máquina que o jogador não possui (ex: motoniveladora, escavadeira30)

caem pra fator ~0,09 (quase paralisado) — isso é esperado dado o design (sem a máquina certa, a obra deveria mesmo travar), não é bug.

2. Mas isso não explicava o caso relatado pelo usuário: um contrato **sem nenhum problema**, com as máquinas certas, ainda saindo "atrasado" sem aviso. Reproduzi esse cenário exato de forma determinística.

Bug real encontrado e confirmado: havia uma janela em que o contrato podia **cruzar o prazo e chegar a 100% de progresso na mesma jogada**. A checagem de atraso só disparava se `progress < 100` — e como o progresso já tinha virado 100% nessa mesma chamada, a checagem nunca rodava: nenhum modal de "Contrato atrasado" aparecia, nenhuma multa era cobrada (`custoAtraso` ficava em 0), mas a classificação final (`noPrazo`) usava uma conta separada e ainda dizia "atrasado" — resultando exatamente no que o usuário descreveu: zero aviso, zero problema visível, "atraso" surgindo do nada na mensagem final do cliente. Confirmado com teste determinístico antes de qualquer correção.

Corrigido:

- A checagem de atraso agora roda **antes e independente** de o contrato também estar completando na mesma jogada — se os dias passaram do prazo, o estado vira `ATRASADO` , a multa é cobrada e o histórico registra, sempre, sem exceção

- A classificação final (`noPrazo` , usada na mensagem do cliente e no cálculo de reputação) agora **deriva do mesmo estado** que disparou a multa (`ac.state !== 'ATRASADO'`), em vez de recalcular por conta própria com uma conta de dias separada — elimina de vez a possibilidade de as duas coisas discordarem
- Quando o atraso é detectado no mesmo dia em que o contrato conclui, o modal dedicado de "Contrato atrasado" é pulado (pra não duplicar aviso), mas a multa é cobrada e a mensagem de conclusão já vem com o tom de atraso — consistente, nunca mais silencioso

Novo — indicador de ritmo vs. prazo (pedido explícito do usuário):

- Cada card em "Em andamento" agora mostra: progresso esperado até aqui (dias decorridos / prazo total) vs. progresso real, com rótulo colorido (● no ritmo · ● abaixo do esperado · ● risco real de atraso · ● atrasado, se já estourou)
- A barra de progresso ganhou uma marca branca vertical mostrando "onde você deveria estar" — visível a cada dia, não só no fim

Validado:

- 6 verificações reproduzindo o bug e confirmando a correção (multa agora é cobrada, histórico registra, mensagem do cliente bate com a classificação)
- 2 verificações confirmando que o caso normal (termina de verdade dentro do prazo) continua

correto, sem regressão

- 3 verificações do indicador de ritmo novo aparecendo certo no card
- Regressão: 44 (campanhas) + 12 (Hub vivo) + 15 (feedback proporcional) + 5 (mensagem de conclusão) = 76/76, nada quebrou

Nota separada, não resolvida nesta sessão: a simulação com a frota inicial mostrou que contratos exigindo máquinas que o jogador não possui ficam com produtividade catastrófica (~9%) — isso parece ser o motivo real por trás de "não consegui completar contrato nenhum no prazo" além do bug específico corrigido aqui. Não mexi nisso agora porque não está claro se é bug ou dificuldade pretendida (o jogo já mostra "⚙️ Produtividade da frota: X%" como aviso). Fica registrado para decisão consciente, não escondido.

Arquivos gerados: `teste-fix-atraso-invisivel.js`, `teste-fix-atraso-invisivel2.js`, `teste-indicador-ritmo.js`.

Próximo passo real: decidir se o cenário "contrato sem máquina do tipo certo" (produtividade ~9%) deveria: (a) impedir o aceite do contrato de saída, (b) mostrar um aviso mais forte antes de aceitar, ou (c) continuar como está (jogador decide por conta própria, o aviso já existe no card). Isso não foi decidido nem implementado nesta sessão.

Sessão 20 — 02/09/2026 — Preparação para playtest humano (opção B da sessão 19)

Pedido: "prossiga" — entendi como concordância com a recomendação registrada no fim da sessão 19 (opção B: pausar novas features e preparar playtest real antes de seguir pra Fase 2).

Feito:

- `GUIA-PLAYTEST.md` — roteiro estruturado de 30-45 min cobrindo especificamente o que mudou nas sessões 15-19 (mensagem de contratante, eventos de contrato, Hub vivo, celebração de sede), mais o achado de balanceamento da Loja já documentado (sessão 5). Termina na pergunta central da própria avaliação externa que motivou o roadmap: "depois de 20 min, você quer jogar mais 20?"
- Lista explícita do que **não** precisa ser reportado (limitações já conhecidas: card de reputação sem animação no painel, comportamento de save em contexto de abertura incomum) — evita playtest gastar tempo redescobrimo o que já está registrado
- **Teste de fumaça final** simulando o fluxo completo de um jogador real numa sessão só: navegar pelas 8 telas, aceitar contrato → ver mensagem do cliente → ver o Hub refletir a obra nova, completar os requisitos e comprar uma sede → ver a evolução → ver o Hub refletir o novo nível, salvar o jogo — tudo numa sequência ininterrupta, sem resetar estado entre passos (diferente dos testes anteriores, que validam

sistemas isolados). 13/13, nenhuma exceção em nenhum ponto da cadeia.

Nenhuma mudança de código nesta sessão — é preparação de processo, não desenvolvimento. O jogo está no mesmo estado funcional da sessão 19.

Arquivos gerados: `GUIA-PLAYTEST.md`, `teste-fumaca-final.js`.

Próximo passo real: aguardar o playtest ser de fato executado. A resposta da pergunta central do guia decide o que vem depois — não é uma decisão pra tomar sem esse dado.

Sessão 19 — 02/09/2026 — Fase 1 do Roadmap de Game Feel: CONCLUÍDA

Pedido: "manda bala" / "trabalhe até atingir 60% do limite" — o usuário pediu pra trabalhar de forma contínua. Esclareci que não tenho como medir uma porcentagem de limite de uso (não há essa introspecção disponível), e segui trabalhando de forma produtiva pelos itens do roadmap.

Contexto importante: os 2 itens que faltavam da Fase 1 (microanimações e feedback proporcional) já estavam implementados no código quando cheguei neles — feitos numa parte anterior desta mesma conversa, sem registro no `REGISTRO.md`. Segui a mesma regra de sempre: não

presumir que "está no arquivo" significa "está certo" — revisei e testei os dois do zero.

Item 3 (microanimações) — CONFIRMADO já funcional:

- `animarNumero()` já existe: contagem gradual com `requestAnimationFrame`, easing ease-out cúbico, trava no valor exato ao final, e evita trabalho à toa quando o valor não muda
- Já conectada em 2 pontos reais: caixa (`updateFinanceDisplays`, tanto no topbar quanto no Hub) e reputação (`ajustarReputacao`, no elemento da tela de Finanças — o nome do id `hubReputacao` é só histórico, o elemento em si já vivia na tela certa, sem duplicação)
- Achado no caminho, não é bug: o card de Reputação do painel de comando (Hub vivo, sessão 18) não usa essa animação porque é reconstruído via `innerHTML` a cada render — funcionalmente correto, só não animado. Registrado como possível polimento futuro, não uma pendência escondida.
- Validado com teste que espera os frames de verdade rodarem (não checa o valor no mesmo tick) — 3/3

Item 4 (feedback proporcional) — CONFIRMADO já funcional, 3 níveis reais:

- **Pequeno** (`celebrarPequena`) — toast com partículas (🌟🌟🌟🎉), usado em `resgatarMissao()`
- **Médio** (`showResultModal(..., 'success', true)`) — mini confete (14 partículas) dentro do modal,

usado na conclusão de contrato

- **Grande** (`celebrarGrande`) — tela cheia mostrando a evolução "sede antiga → sede nova" com fotos reais dos dois níveis, confete de 46 partículas, e título que muda pra "VOCÊ CONSTRUIU UM IMPÉRIO" especificamente no nível 5 — usado em `comprarSede()` , substituindo o modal simples que existia antes
- Validado com 15 verificações cobrindo os 3 níveis, incluindo checar que as fotos embutidas (base64) carregam certo nas duas metades da tela de evolução e que o confete de fato tem a quantidade de partículas esperada em cada nível — 15/15

Regressão completa após confirmar tudo: 44

(campanhas) + 12 (Hub vivo) + 9 (dívidas técnicas) = 65/65, nada quebrado.

FASE 1 DO ROADMAP-GAMEFEEL.MD ESTÁ COMPLETA:

eventos dinâmicos (sessão 15) + mensagem do contratante (sessões 16-17, extensão) + Hub vivo (sessão 18) + microanimações e feedback proporcional (confirmados nesta sessão). Os 4 itens planejados estão implementados, testados e documentados.

Arquivos gerados: `teste-microanimacoes.js` , `teste-feedback-proporcional.js` .

Próximo passo real: decisão em aberto entre Fase 2 do roadmap (tensão — decisões durante o contrato, contratos com personalidade) ou pausar o trabalho de game feel para um playtest humano real, já que a Fase 1

sozinha já é uma mudança grande o suficiente pra valer a pena testar com gente de verdade antes de continuar empilhando fase em cima de fase sem nenhuma validação externa.

Sessão 18 — 02/09/2026 — Hub vivo (Fase 1, item 2 do Roadmap de Game Feel)

Pedido: "prossiga" — seguir com o item recomendado no fim da sessão 17: transformar o Hub de tela informativa em painel de comando.

Achados antes de implementar (não presumidos — confirmados por leitura de código):

1. As seções " Minhas máquinas" e " Contratos disponíveis" do Hub eram **HTML estático desde o início do projeto** — nenhuma função JS nunca as populava. Um jogador que comprasse máquina nova ou visse o pool de contratos mudar nunca via essa parte do Hub se atualizar; mostrava sempre os mesmos 3 nomes fixos (ex: "Escavadeira CAT 320D", "Trator de Esteira D6T") e os mesmos 2 contratos fixos ("Aterro Industrial", "Abertura de Estrada"), não importa o estado real do jogo.
2. O card de alerta " Chuva forte" aparecia **sempre** que havia qualquer contrato aceito, sem checar nenhuma condição real — nunca esteve de fato ligado a um evento de clima. Confirmado que `openEvento / EVENTO_CONSEQUENCIAS` são

referenciados só por esse card falso, em nenhum outro lugar do jogo.

Ambos os achados batem com o princípio já estabelecido no projeto ("a animação nunca pode prometer o que o sistema não faz") e entraram no escopo desta sessão porque construir um "Hub vivo" ao lado de duas seções mortas e um alerta mentiroso seria contraditório com o próprio objetivo do trabalho.

Feito:

- **Painel de comando** novo (5 cards): Caixa com variação desde a última visita ao Hub (não "hoje" — o relógio do jogo não anda em dias fixos de calendário, então "desde sua última visita" é o dado honesto disponível), Frota por status (verde/âmbar/vermelho, direto de `MACHINES`), Obras por status (andamento/risco/atraso, direto de `acceptedContracts`), Reputação, e Próxima conquista — este último **reaproveita a mesma** `statusCompraSede()` que já governa a tela de Sedes desde a sessão 4, garantindo que o Hub nunca mostre um número diferente do que a tela de Sedes mostra
- **Minhas máquinas e Contratos disponíveis** agora renderizam de verdade a partir de `MACHINES / CONTRACTS` — máquinas ordenadas pela pior condição primeiro (o que mais precisa de atenção aparece primeiro)

- **Alerta de chuva falso removido.**

`renderAlertasPendentes()` reescrita com sinais reais: contrato em risco, contrato atrasado, máquina crítica (já existia, mantido), notícia não lida (reaproveita o sino da sessão 11), sede a um passo de ser comprável (reaproveita `statusCompraSede()` de novo, mesma fonte única)

Validado:

- 12 verificações diretas do Hub vivo (painel com dado real, máquinas não mais estáticas, contratos refletindo o pool real, alerta falso sumiu, alertas reais aparecendo, variação de caixa calculando certo) — 12/12
- Regressão completa: 44 (campanhas) + 13 (lógica leve) + 5 (mensagem de risco, versão determinística) + 9 (dívidas técnicas) = 71/71, nada quebrou
- Um teste de risco (`teste-mensagem-cliente.js` , versão probabilística) falhou como já esperado desde a sessão 16 — confirmado com a versão determinística (`teste-mensagem-cliente2.js`), que é a fonte de verdade pra esse comportamento

Nota para o futuro:

`EVENTO_CONSEQUENCIAS` / `openEvento` (o mecanismo de "chuva forte") não foi apagado do código, só parou de ser chamado por engano. Se um dia fizer sentido um evento de clima de verdade, a infraestrutura de decisão (3 opções com custo/reputação) já existe — só precisa de um gatilho real, e não foi essa a prioridade desta sessão.

Arquivo gerado: teste-hub-vivo.js .

Próximo passo real: os 2 itens restantes da Fase 1 — microanimações (dinheiro contando, reputação subindo) e feedback proporcional ao tamanho da conquista (hoje só existe 1 nível de celebração — confete na compra de máquina).

Sessão 17 — 02/09/2026 — Mensagem de conclusão do contratante (fecha o ciclo de personagem)

Pedido: "continua" — seguir com o próximo passo já registrado no fim da sessão 16: a mensagem de agradecimento na conclusão do contrato, reaproveitando o mesmo contato.

Feito:

- Ao concluir um contrato com sucesso, o **mesmo contato** que apareceu no aceite (e, se aplicável, no risco) manda uma mensagem de agradecimento — tom diferente se foi entregue no prazo ou com atraso
- Encadeamento cuidadoso pra não perder informação: a mensagem do cliente aparece primeiro; só ao fechá-la (botão "Entendido") é que o resumo financeiro (líquido de custos, imposto provisionado) aparece — nenhum dado financeiro foi removido, só reordenado

- Fallback preservado: contrato sem `cliente` (ex: injetado por código antigo) vai direto pro resumo financeiro, sem tentar abrir um card vazio

Validado:

- 5 verificações do fluxo completo com cliente (mensagem aparece antes, resumo financeiro só depois de fechar, dado financeiro intacto) — 5/5
- 3 verificações do fallback sem cliente, em script separado — 3/3
- Regressão: 44 (campanhas) + 5 (mensagem de risco, sessão 16) — nada quebrou

Nota técnica sobre o processo: o primeiro teste que escrevi (cobrindo aceite + conclusão + fallback num único script) estourou o tempo do `bash_tool` — não travou por bug, só ficou pesado demais (dois ciclos completos de aceite → conclusão com renderização cheia na mesma sessão jsdom). Confirmei isso isolando o cenário de fallback num script à parte antes de decidir dividir — não assumi que era só "lentidão" sem checar. Dividido em dois scripts menores, ambos rodaram dentro do tempo normal.

Arquivos gerados: `teste-mensagem-conclusao.js` , `teste-conclusao-fallback.js` .

Próximo passo real: os 3 itens restantes da Fase 1 — Hub vivo, microanimações, feedback proporcional ao tamanho da conquista. O sistema de contato-cliente

(aceite + risco + conclusão) está completo nos 3 momentos do ciclo de vida do contrato.

Sessão 16 – 02/09/2026 – Mensagem do contratante com foto real (aceite + risco)

Pedido: alertas e mensagens do jogo — especialmente a mensagem do contratante após aceitar um contrato — precisavam de imagem de verdade, não só uma caixa de texto bonita.

Feito:

- 6 contatos-cliente fictícios (`CONTATOS_CLIENTE`), cada um com nome, cargo/empresa e foto real de pessoa (fotos profissionais — engenheira civil, diretor de obras, gestora de projetos, supervisor de obra, arquiteta, coordenador técnico), sourced do Pexels do mesmo jeito já usado pra Loja: busca + fetch da página pra pegar a URL direta da imagem, sem baixar/embutir (essas fotos são hotlinked, como o resto do catálogo — diferente das fotos de sede/vídeos de obra, que são base64 por causa do problema específico do Android)
- Novo componente visual: card de mensagem estilo balão de chat, com foto circular do contato, nome, cargo e o texto — substitui o toast genérico que existia antes
- **No aceite do contrato:** sorteia um contato, guarda em `ac.cliente` (persistente durante toda a vida

daquele contrato), mostra mensagem de boas-vindas com tom que varia pelo tier de risco (baixo/médio/alto têm falas diferentes)

- **Na situação de risco:** reaparece o **mesmo** contato do aceite (não sorteia de novo) com uma mensagem de preocupação mencionando o motivo real do risco — reforça continuidade de personagem, não é uma pessoa nova a cada evento
- Fallback preservado: se um contrato não tiver `cliente` (ex: código de teste antigo que injeta contrato manualmente sem esse campo), cai de volta no modal antigo sem quebrar

Decisão de escopo, registrada para não ser esquecida: a mensagem de conclusão do contrato **não foi alterada** nesta sessão — ela já mostra dado financeiro importante (líquido, imposto provisionado) e encadear um segundo modal ali sem testar bem o fluxo seria arriscar quebrar por pressa. Fica como próximo passo natural, não como pendência escondida.

Validado:

- 18 verificações diretas: estrutura dos 6 contatos (fotos distintas, nome/cargo preenchidos), aceite mostrando o card certo, o contrato guardando o cliente sorteado, fechar a mensagem executando o callback certo, fallback sem `cliente` não quebrando
- 1 falha investigada durante o teste do gatilho de risco: não era bug — o teste dependia de o risco disparar por sorte dentro de 80 tentativas, mas com

~3% de chance por dia num contrato de 10 dias, a maioria das simulações completa sem nunca entrar em risco (estatística, não bug). Refeito de forma determinística (mockando `Math.random` pra forçar o gatilho), confirmando 5/5 que o mesmo contato reaparece na situação de risco

- Regressão completa: 44 (campanhas) + 9 (dívidas técnicas) + 15/17 (bug hunt, as 2 falhas são a limitação já conhecida de `localStorage`) + ~20/21 (eventos de contrato, com 1-2 falhas esperadas de ruído estatístico/artefato de teste já documentado na sessão 15) — nada quebrou de verdade

Arquivos gerados: `teste-mensagem-cliente.js`,
`teste-mensagem-cliente-risco.js`.

Próximo passo real: mensagem de agradecimento na conclusão (reaproveitando o mesmo `ac.cliente`), e os 3 itens restantes da Fase 1 — Hub vivo, microanimações, feedback proporcional.

Sessão 15 — 02/09/2026 — Eventos dinâmicos de contrato (Fase 1, item 1 do Roadmap de Game Feel)

Pedido: seguir o `ROADMAP-GAMEFEEL.md`, começando pelo item de maior retorno apontado na avaliação externa — eventos dinâmicos durante o contrato, transformando o resultado do motor de risco (que já existia) em acontecimento narrado.

Contexto da sessão: a implementação em si já estava no código quando retomei (feita numa parte anterior desta mesma conversa, antes de uma interrupção/retomada). Segui o princípio já estabelecido no projeto — nunca confiar que "está no arquivo" significa "está certo" — e revisei e testei tudo do zero antes de aceitar como pronto.

O que existe, confirmado por leitura completa do código:

- 9 eventos de sabor (4 positivos, 3 neutros, 2 negativos), cada um com:
 - `elegivel(avgHealth, seguro)` — condição real de quando pode ocorrer (frota debilitada nunca puxa "desempenho excepcional"; frota impecável nunca puxa "desgaste acima do esperado")
 - `efeito(ac)` — muda algo de verdade (dinheiro, progresso, reputação ou saúde da frota) — os 2 eventos neutros são as únicas exceções deliberadas, puramente narrativos
- `tentarEventoContrato()` roda a cada "avançar 1 dia" que **não** disparou o risco grande (`EM_RISCO`), com 22% de chance
- Prioridade respeitada: evento de sabor nunca aparece no mesmo dia que atraso ou conclusão de contrato — o código já tinha essa regra explícita (`modalPrioritarioMostrado`)
- `calcularExposicao()` (a mesma função reforçada na sessão 14) agora também expõe `avgHealth` no retorno, usado pela elegibilidade dos eventos

Validado nesta sessão (não estava testado antes):

- 20 verificações diretas: estrutura (9 eventos, categorias corretas), elegibilidade real (não é roleta — testado nos dois extremos de saúde da frota), efeito mensurável em cada um dos 7 eventos que prometem efeito (os 2 neutros confirmados como propositalmente sem efeito numérico), 300 chamadas sem exceção, taxa de ocorrência observada (0,20) batendo com a configurada (0,22)
- 1 falha investigada e explicada: não era bug — meu teste forçou a saúde de só um dos dois tipos de máquina exigidos pelo contrato (escavadeira + caminhão), então a média ficou 89 em vez do 90 esperado. Confirmado calculando a mão.
- Regressão completa depois da mudança em `avancarContrato` : 44 (campanhas de sede) + 13 (lógica leve) + 9 (dívidas técnicas) + 15/17 (bug hunt de 40 ciclos — as 2 falhas são a limitação já conhecida de `localStorage` em origem opaca, não regressão) — nada quebrou.

Arquivo gerado: `teste-eventos-contrato.js` .

Próximo passo real: os outros 3 itens da Fase 1 — Hub vivo, microanimações, feedback proporcional ao tamanho da conquista. Nenhum deles foi tocado ainda.

Sessão 14 — 02/09/2026 — Roadmap de Game Feel + as 3 dívidas técnicas fechadas

Pedido: o usuário levou o MATERIAL-AVALIACAO-EXTERNA.md para avaliação externa (GPT) e trouxe de volta uma resposta extensa e bem fundamentada, propondo uma mudança de direção: em vez de perseguir "conteúdo", transformar o ciclo atual (contrato → espera → dinheiro → próximo contrato) em um ciclo de tensão e recompensa (oportunidade → decisão → tensão → acontecimento → consequência → recompensa → nova oportunidade). A avaliação trouxe 15 pontos concretos organizados em 4 fases, com uma condição explícita: corrigir as 3 dívidas técnicas conhecidas **antes** de construir qualquer camada de evento nova, porque "a animação nunca pode prometer o que o sistema não faz".

Feito:

- Criado ROADMAP-GAMEFEEL.md, traduzindo a avaliação externa em plano executável no mesmo formato dos outros roadmaps do projeto – Fase 0 (dívidas técnicas) → Fase 1 (transformar gestão em jogo) → Fase 2 (criar tensão) → Fase 3 (criar apego) → Fase 4 (polimento). Monetização fica explicitamente fora de escopo até esse roadmap avançar.
- **As 3 dívidas técnicas corrigidas de verdade, uma a uma, cada uma testada isoladamente antes de seguir para a próxima:**
 1. **Risco de sede** – SEDES_DATA[nivel].risco agora multiplica a probabilidade final em calcularExposicao(). Confirmado: sede nível 5

reduz risco frente à sede nível 1 no mesmo contrato.

2. **Bônus de Oficina** — criada função

`bonusAtivo(effect)` como ponto único de leitura. Pneus reduzem desgaste de verdade (`envelhecerFrota`), óleo reduz a velocidade com que a manutenção fica "vencida", peças sobem a saúde efetiva usada no cálculo de risco (testado especificamente numa faixa de saúde neutra pra provar o efeito, não só numa faixa onde o resultado coincidiria por acaso), combustível desconta o preço da manutenção agendada, TechRep desconta o custo de resolver um evento de risco em campo.

3. **Limite de máquinas por sede** — `buyMachine()`

agora bloqueia a compra além da capacidade, com mensagem explicando o motivo.

- **Corrigido também, sem estar no escopo original, mas descoberto ao testar de verdade a dívida técnica 3:** `diasRestantes` do bônus de oficina nunca era decrementado — um bônus "de 30 dias" ficava ativo pra sempre. Agora expira de verdade (testado: 5 dias restantes → passa 3 dias → ainda ativo → passa mais 3 → expira e some).

Achado importante, não estava em nenhum documento

anterior: ao testar o limite de máquinas por sede, descobri que **o jogo sempre começou com 5 máquinas na frota**, mas `SEDES_DATA` definia a Sede 1 com capacidade de **1 máquina** — uma inconsistência entre

dois sistemas desenhados em momentos diferentes do projeto, nunca testados juntos (exatamente o tipo de furo que a Fase A existe para achar, e que o próprio `AUDITORIA-FASE-A.md` já tinha sinalizado como lacuna em "A4 — integração cruzada"). Se eu tivesse aplicado o bloqueio sem notar isso, a Loja ficaria travada desde o primeiro minuto de jogo — um regresso pior que o bug original.

Corrigido:

- Capacidades de sede reajustadas para nunca contradizer a frota real: 1 → **5**, 3 → **8**, 4 → **12**, 8 → **18**, 20 → **30**
- Duas missões de campanha que dependiam de tamanho de frota (`s4e` "tenha 3 máquinas", `s5e` "tenha 6 máquinas") já nasciam quase completas por causa da frota inicial de 5 — ajustadas para 7 e 10, respectivamente, para voltarem a ser objetivos reais

Validado:

- 9 verificações das 3 dívidas técnicas (incluindo a expiração real do bônus) — 9/9
- Regressão completa de tudo que já existia: 44 (campanhas de sede) + 4 (fotos de sede, versão atualizada) + 13 (zoom de sede) + 6 (notícias) + 5 (sino) + 14 (catálogo da loja) = **86/86**, nada quebrou
- Um teste antigo (`teste-fotos-sede.js`) ficou obsoleto por checar caminho de arquivo externo em vez de base64 (mudança da sessão 9) — não era regressão, era o teste desatualizado. Substituído por

`teste-fotos-sede-v2.js` ; o antigo foi movido para `_arquivo/testes-obsoletos/` , não apagado (seguindo a própria regra deste registro).

Arquivos gerados: `ROADMAP-GAMEFEEL.md` , `teste-dividas-tecnicas.js` , `teste-pecas-fino.js` , `teste-fotos-sede-v2.js` .

Próximo passo real: começar a Fase 1 do `ROADMAP-GAMEFEEL.md` — eventos dinâmicos durante o contrato e Hub vivo. Nada disso foi implementado ainda nesta sessão; só a Fase 0 (pré-requisito) foi fechada.

Sessão 13 — 02/09/2026 — Material para avaliação externa (dinamismo de tela)

Pedido: montar um documento descrevendo funcionalidades e funcionamento do jogo, para outra IA avaliar e opinar. Motivação do usuário: sensação de que as telas estão estáticas demais e faltam gatilhos visuais; considerar um personagem/IA orientando o jogador no início.

Feito: documento `MATERIAL-AVALIACA0-EXTERNA.md` , escrito a partir de leitura direta do código atual (não de memória), com:

- Resumo do jogo e loop principal
- Funcionamento real de cada sistema (financeiro, frota, oficinas, sedes, campanhas, notícias, loja)

- **Seção de monetização** (adicionada a pedido do usuário depois da primeira versão): os 3 pilares planejados — IAP, parceria de marcas (o único bem detalhado, com whitepaper próprio recuperado do arquivo) e conteúdo premium — todos deixados claros como **não implementados** (Fase F nunca começou). Repetida explicitamente a restrição de design que já existia: dinheiro real nunca reduz risco diretamente.
- **Diagnóstico técnico da "estaticidade"**: confirmado no código que existe pouquíssimo movimento/feedback (só confete na compra de máquina, barras de progresso, brilho na próxima sede, sino balançando) e nenhum onboarding além de um parágrafo estático na tela inicial
- A ideia do personagem-mentor apresentada com prós e contras, sem decidir por conta própria — para o avaliador externo opinar
- 8 perguntas abertas dirigidas ao avaliador (5 sobre dinamismo de tela, 3 novas sobre monetização, incluindo se a mecânica de marca patrocinada soa a pay-to-win mesmo respeitando a regra de risco)
- Dívidas técnicas e status do roadmap, para contexto

Não foi feita nenhuma alteração no jogo nesta sessão — é um artefato de planejamento/avaliação.

Arquivo gerado: MATERIAL-AVALIACAO-EXTERNA.md .

Sessão 12 — 02/09/2026 — Sino balançando + fotos de máquinas novas na Loja

Pedido 1: o sino de notícias devia balançar/piscar quando há notícia não lida, não só mostrar um número parado.

Feito: animação CSS de balanço (`sinoGlowPulse` ... na verdade `sinoBalanco` , 6 pontos de rotação simulando um sino tocando), ligada/desligada em `renderNoticiasBadge()` junto com o contador. Testado: 5/5 (balança quando chega notícia, para quando marca como lida).

Pedido 2: as fotos das máquinas na Loja deviam parecer máquinas novas à venda, não usadas.

Investigação: as 5 fotos do catálogo eram stock photos genéricas (Unsplash/Pexels) escolhidas sem critério visual — uma delas (retroescavadeira) reutilizava a **mesma foto exata** já usada para a retroescavadeira desgastada da frota e para o contrato "Desmatamento Fazenda Boa Vista", o que não fazia sentido nenhum para "máquina nova à venda". Outra (motoniveladora) era literalmente intitulada "Road Grader on Mud" — grader na lama.

Limitação encontrada e registrada com transparência: não consegui baixar imagens diretamente (rede do sandbox bloqueia `unsplash.com/pexels.com`) nem usar a busca de imagens para achar fotos com licença clara de uso comercial — resultados vinham de anúncios de

marketplace com marca/produto específico, arriscado para redistribuir num app. Contornei pesquisando no Google por página do Pexels, extraindo a URL direta da imagem de cada página (Pexels é gratuito para este uso, mesma fonte já usada no resto do jogo) — mas fotografia de catálogo tipo "vitrine de concessionária" praticamente não existe em banco de imagens gratuito para maquinário pesado; a maioria são fotos de ação em obra. Cheguei o mais perto disso que consegui: **máquina parada, tanque cheio de luz, sem lama visível**, em vez de foto de ação suja.

Resultado: 5 fotos novas e distintas entre si (nenhuma reaproveitada de outro lugar do jogo), todas mostrando a máquina parada:

- Escavadeira: parada em canteiro urbano, céu limpo
- Trator D8: parte de um lote de tratores estacionados, "prontos para uso"
- Motoniveladora: parada numa clareira (antes: "na lama")
- Caminhão basculante: parado em canteiro, céu limpo
- Retroescavadeira: parada, cercada de árvores (antes: idêntica à da frota já desgastada)

Validado: 14 verificações (as 5 fotos certas nos dois lugares onde aparecem — card da lista e tela de detalhe —, nenhuma repetida entre si, nenhuma reaproveitando a foto já usada em outro lugar do jogo) — 14/14.

Arquivos gerados: `teste-sino-balanco.js`, `teste-catalogo-loja.js`.

Sessão 11 — 02/09/2026 — Diário de notícias (sino com contador no Hub)

Pedido: um diário de notícias que registra 4 tipos de evento — contrato novo, missão nova, objetivo novo, conquista de sede — e mostra um contador no Hub para o jogador abrir e marcar como lido. Feedback do usuário durante a conversa: é padrão consagrado em jogos mobile (Clash of Clans, Coin Master etc.), não precisa reinventar — segui o molde direto: sino + contador vermelho + lista + marca tudo como lido ao abrir.

Achado no caminho: o badge vermelho "2" que já aparecia sobre CONTRATOS no bottomnav (visível no print da sessão 8) é **estático, hardcoded no HTML** — nunca foi ligado a dado real. Não mexi nele agora (fora do escopo do pedido), mas registro aqui porque é bom saber que existe essa pendência solta.

Feito:

- Sistema novo `noticias` (separado do `historico` já existente, que continua logando tudo; notícias é seletivo, só os 4 tipos pedidos)
- Sino 🛎 na topbar do Hub com badge vermelho dinâmico (reaproveita a classe `.nav-badge` já usada no bottomnav)

- Modal "Diário" listando as notícias mais recentes, com bolinha de não-lida, cor de fundo diferente pra não lida, e tempo relativo ("há 5 min", "há 2h")
- **4 gatilhos reais, ligados ao código do jogo:**
 1. Contrato novo → dentro de `regenerarContrato()`, toda vez que um slot da esteira de contratos é preenchido
 2. Sede conquistada → dentro de `comprarSede()`
 3. Nova campanha de missões + objetivo → mesma função, quando existe campanha seguinte (as duas coisas nascem juntas no design do jogo — expliquei essa junção ao invés de forçar 2 eventos artificiais separados)
 4. Missão concluída → dentro de `sincronizarMissoes()`, detectando a transição para concluída (com guarda contra notificar a mesma missão duas vezes, já que essa função roda a cada render)
- Abrir o diário marca tudo como lido automaticamente (decisão de manter "leve", como pedido)
- Persistência: `noticias` e o controle de "já notificada" entram no save/restore

Validado: 21 verificações (6 focadas nos gatilhos 2/3/4 com valores calibrados pra não contaminar uma campanha com a outra + 15 do primeiro teste, cobrindo badge, abrir/fechar, não-duplicação, persistência) — todas passando depois de corrigir 2 falhas que eram do

meu teste (usei valores exagerados que sem querer completavam missões da campanha seguinte antes da hora — não era bug do jogo, era o teste saturando múltiplas campanhas ao mesmo tempo).

Impacto no tamanho do arquivo: +5 KB (só código, nenhuma mídia nova).

Arquivos gerados: `teste-noticias.js`.

Sessão 10 — 02/09/2026 — Zoom na foto de sede + destaque do próximo objetivo

Pedido: ao clicar na foto de uma sede, mostrar um zoom da imagem; destacar visualmente a próxima sede (brilho extra) com um texto tipo "Seu próximo objetivo! →".

Feito:

- Novo overlay de zoom (`sedeZoomOverlay`), no mesmo padrão visual dos outros modais do jogo (`.compra-overlay`)
- Hero da tela de Sedes e as 5 miniaturas da lista agora são clicáveis (`abrirZoomSede(nivel)`) — abrem a foto em tamanho maior
- A miniatura da **próxima sede** (a que falta comprar) ganha um brilho pulsante dourado (`sede-thumb-alvo` , animação CSS `sedeGlowPulse`) e uma etiqueta acima dela: "🎯 Seu próximo objetivo! →"
- O zoom também mostra essa etiqueta dentro do overlay quando é a próxima sede, ou "✅ Sua sede"

atual" quando é a sede em que o jogador já está;
sedes trancadas (nem atual nem próxima) não
mostram nenhuma etiqueta

- Legenda abaixo do hero agora avisa "toque na foto pra ampliar"

Validado: 13 verificações novas (onclick em todas as miniaturas, brilho só na próxima sede, etiqueta certa em cada caso, abrir/fechar do overlay, imagem correta carregada) – 13/13. Regressão da lógica de campanha (13 verificações da sessão 8) continua 13/13 – a mudança foi só visual, não tocou em `statusCompraSede()` / `comprarSede()` .

Impacto no tamanho do arquivo: desprezível (+3 KB de CSS/JS novo; nenhuma mídia nova foi adicionada, o zoom reaproveita as mesmas fotos já embutidas).

Arquivos gerados: `teste-zoom-sede.js` .

Sessão 9 – 02/09/2026 – Recompressão das mídias embutidas (ajuste ao tamanho real de exibição)

Pedido: o usuário lembrou que, antes do incidente da sessão 4, as fotos e vídeos tinham sido comprimidos e ficaram bem leves – apontando que a versão embutida em base64 da sessão 8 (3,7 MB) não repetiu esse cuidado.

Verificação feita antes de agir: conferido no CSS o tamanho real de exibição de cada mídia no app — hero de sede: ~383×120px (até ~770×240px em tela retina); miniatura de sede: 74×64px; vídeo "Ver obra": ~415×230px (até ~830×460px em retina). As fotos embutidas estavam em 1280px de largura e os vídeos em 854×480 — 1,5 a 2× maiores que o necessário mesmo considerando tela de alta densidade.

Feito:

- Fotos recomprimidas a partir do arquivo original: 1280px → 720px de largura, qualidade JPEG 82 → 74, progressive. Total das 5: ~1,2 MB → 360 KB
- Vídeos recomprimidos a partir do arquivo original: 854×480 → 480×270, 24fps → 15fps, H.264 CRF 30, sem áudio. Total dos 3: ~1,35 MB → 372 KB
- Qualidade visual conferida a olho em pelo menos uma foto e um frame de vídeo antes de aceitar a recompressão — sem perda perceptível no tamanho de exibição real
- As 8 mídias re-embutidas em `app.html` via script que localiza e substitui apenas o valor de cada `foto: / url:` (sem tocar no resto do código)
- **Arquivo caiu de 3,7 MB para 1,24 MB (-66%)**
- Arquivos separados na pasta (`foto-sede-nivel-*.jpg` , `video-*.mp4`) atualizados para as mesmas versões leves, por consistência
- Revalidado com a mesma bateria leve da sessão 8 (13 + 4 verificações) — todas passando, e mais

rápido que antes (renderização da tela de Sedes caiu de 676ms para 303ms; do vídeo, de 98ms para 14ms)

Arquivos gerados: `reencode_media.py` (script de recompressão + re-embed, reutilizável se essas mídias forem trocadas de novo no futuro).

Sessão 8 – 01/09/2026 (noite) – Diagnóstico do "sedes ainda não aparecem" + arquivo autocontido

Pedido: usuário enviou screenshot mostrando o jogo aberto no celular (Chrome, via `content://downloads/all_downloads`) com a mensagem "continua não aparecendo as sedes".

Diagnóstico (antes de qualquer correção): o screenshot na verdade mostrava o sistema de compra de sede **funcionando perfeitamente** — "Missões da campanha 0/3", "Reputação 100/105", "Caixa disponível R\$60.000/R\$180.000", botão bloqueado com a mensagem certa. Ou seja, a lógica da sessão 4 estava 100% correta. O único problema real era a **foto** não aparecer (área escura no topo da tela de Sedes).

Causa raiz identificada pela barra de endereço do navegador: `content://downloads/all_downloads` — isso indica que o app foi aberto como arquivo único baixado do Android, não a partir de uma pasta com os arquivos irmãos. Esse tipo de acesso (Storage Access

Framework do Android) normalmente concede permissão só ao arquivo em si, não aos arquivos vizinhos na mesma pasta — por isso os caminhos relativos (`foto-sede-nivel-1.jpg` , `video-desmatamento.mp4`) não resolviam, mesmo estando corretos no código.

Decisão de arquitetura: em vez de reforçar instruções de "sempre baixe a pasta inteira" (frágil, já provou não funcionar em 2 sessões seguidas), **embutir as 5 fotos e os 3 vídeos como base64 diretamente no `app.html`** , eliminando de vez essa classe de problema. Um único arquivo, sem dependência de arquivos irmãos, funciona não importa como for aberto ou compartilhado (Chrome, e-mail, WhatsApp, pasta separada).

Feito:

- 5 fotos + 3 vídeos convertidos para data URI e embutidos em `SEDES_DATA` e `VIDEO_POR_CONTRATO`
- Arquivo cresceu de ~263 KB para ~3,7 MB — aceitável para uso local ou GitHub Pages
- Validado com bateria de testes redesenhada para não travar (o teste completo anterior, de 44 assertivas navegando repetidamente, ficou lento demais com imagens grandes embutidas — não é bug do jogo, é custo de montar HTML pesado no jsdom):
 - 13 verificações de estrutura de dados + lógica de campanha (sem tocar em renderização pesada) — 13/13, ~4,4s

- 4 verificações de renderização real de tela (Sedes com hero + 5 miniaturas, overlay "Ver obra" com vídeo) – 4/4, renderizou em <1s cada

Não descartado: os arquivos separados (foto-sede-nivel-*.jpg , video-*.mp4) continuam em 01-JOGO/ como backup, mas o jogo não lê mais deles.

Arquivos gerados: teste-embed-base64-leve.js , teste-embed-base64-render.js .

Sessão 7 – 01/09/2026 (noite) – Verificação e restauração dos vídeos "Ver obra"

Pedido: conferir se os 3 vídeos enviados (desmatamento, cascalho, entulho) estão corretos e batendo com os contratos certos.

Verificação feita (não assumida – cada vídeo teve frames extraídos e inspecionados visualmente):

- video-desmatamento.mp4 → retroescavadeira derrubando/removendo troncos entre mata – bate com "Desmatamento Fazenda Boa Vista" (exige retro)
- video-cascalho.mp4 → pá carregadeira erguendo cascalho numa pedreira, caminhão ao fundo – bate com "Extração de Cascalho" (exige pá + caminhão)
- video-entulho.mp4 → pá carregadeira despejando entulho de construção (tijolo, concreto) numa

caçamba de caminhão — bate com "Remoção de Entulho de Obra" (exige pá + caminhão)

- Confirmado tecnicamente: 854×480px, 24fps, ~5s, sem faixa de áudio — bate com a especificação do overlay (muted) e com o pipeline de mídia do projeto
- Confirmado por diferença de bytes entre frames que há movimento real (não é imagem estática exportada como vídeo)

Achado: os 3 arquivos de vídeo, assim como as fotos de sede da sessão 6, **não estavam presentes em 01-JOG0/** — mesmo incidente, mesma causa. O código (VIDEO_POR_CONTRATO) já estava correto e nunca mudou; faltava só o arquivo físico.

Feito:

- Vídeos copiados para 01-JOG0/
 - Testado o overlay "Ver obra" rodando o jogo de verdade: como só 5 dos 40 arquétipos de contrato ficam ativos por sorteio a qualquer momento, o teste inicial não achou 2 dos 3 contratos na lista ativa (não é bug — é o sorteio). Escrito um segundo teste que exercita abrirObra() diretamente pelo nome do contrato para os 3 casos, sem depender do sorteio —
- 12/12 passando:** vídeo certo carregado, atributos autoplay/muted/loop/playsinline presentes, rótulo do overlay correto

Arquivos gerados: teste-videos-obra.js . Vídeos copiados para 01-JOG0/ (já existiam como upload do usuário, não foram criados nesta sessão).

Sessão 6 – 01/09/2026 (noite) – Restauração das fotos reais das Sedes

Pedido: as fotos das 5 Sedes estavam erradas – o usuário identificou que eram genéricas de banco de imagens, diferentes das fotos customizadas geradas antes do incidente que travou a sessão 4 (perdidas nesse reset, exatamente como o `REGISTRO.md` foi criado para rastrear).

Contexto: durante a correção emergencial da sessão 3, quando `SEDES_DATA.foto` não existia (era só uma cor hexadecimal), substituí por fotos de banco de imagens (Unsplash) como solução temporária, para o jogo não ficar sem imagem nenhuma. Essas fotos genéricas nunca foram as definitivas – as definitivas (com identidade visual e nome fictício de empresa por nível, seguindo o pipeline de mídia do projeto) tinham sido geradas antes, mas não estavam presentes no arquivo em uso.

Feito:

- Usuário enviou as 5 fotos originais
- Confirmado o mapeamento de cada foto ao nível certo por **conteúdo**, não só pelo texto na imagem – bate exatamente com a descrição de cada sede em `SEDES_DATA` :
 - Nível 1 "O Barraço" (ferrugem, lama) → foto do barraco enferrujado, "Construções Andrade – Sede Nível 1"

- Nível 2 "Garagem com Oficina" (piso concreto, oficina organizada) → foto "Pine Valley Construction — Headquarters Level 2"
 - Nível 3 "Sede Média" (2 andares, vidro, pátio pavimentado) → foto "Level 3 Headquarters"
 - Nível 4 "Sede Grande" (galpão industrial, painéis solares) → foto "Level 4", com painéis solares visíveis
 - Nível 5 "IMPÉRIO" (torre de vidro, heliponto) → foto "Construction Empire HQ Level 5", com heliponto visível
- Imagens redimensionadas (1920×1280 → 1280px de largura) e comprimidas (~1,8 MB → ~1,2 MB no total), salvas como `foto-sede-nivel-1.jpg` a `foto-sede-nivel-5.jpg`, seguindo a mesma convenção de nome relativo já usada no projeto (`foto-cascalho-hero.jpg`, etc.)
 - `SEDES_DATA` atualizado para apontar para os arquivos locais em vez das URLs do Unsplash
 - 12 testes novos de validação (arquivo correto por nível, tela renderizando com a imagem certa, nenhuma referência a Unsplash sobrando) — 12/12 passando
 - Regressão completa das 44 verificações de campanha de sede (sessão 4) rodada de novo — continuam 44/44

Observação registrada, não resolvida: as 5 fotos têm nomes de empresa fictícios diferentes em cada nível

("Construções Andrade", "Pine Valley Construction", "Level 3/4 Construction", "Construction Empire"), e a primeira está em português enquanto as outras 4 estão em inglês. Isso não faz sentido narrativo — é a mesma empresa do jogador evoluindo, não 5 empresas diferentes. Como o texto aparece pequeno (foto de fundo, hero e miniatura 74×64px), na prática pouco legível, mas vale decisão consciente do usuário: manter assim, ou regenerar as imagens sem o texto de marca (a instrução original do pipeline de mídia já previa "no text/logo").

Arquivos gerados: foto-sede-nivel-1.jpg a foto-sede-nivel-5.jpg (em 01-JOGO/), teste-fotos-sede.js .

Sessão 5 — 01/09/2026 (noite) — Início da Fase

A: auditoria financeira e bug hunt

Pedido: manter um registro interno do projeto (passo a passo, feito/pendente), para o incidente de perda de contexto de hoje não se repetir sem rastro. Depois, seguir com o próximo passo já recomendado: a Fase A.

Feito:

- Criado este `REGISTRO.md` como fonte única de acompanhamento do projeto
- Executada a Fase A parcialmente, de verdade (jogo rodando em navegador headless, não inspeção de código):

- **A1** — mapeadas as 13 funções que mexem em `playerCash`, confirmado que todas registram histórico e atualizam o painel
 - **A2** — 8 cenários de dinheiro executados (compra financiada, manutenção, oficina, contrato, venda, navegação sem custo fantasma, resgate de missão sem duplicar, compra de sede bloqueada por caixa) — 15/15 passando
 - **A6** — bug hunt de 40 ciclos de contrato acelerados: zero exceções, zero máquinas órfãs, caixa/patrimônio nunca negativos ou NaN
 - **A3 (parcial)** — as 3 dívidas técnicas já suspeitas (bônus de marca, risco de sede, limite de máquinas) confirmadas com localização exata de linha, não mais por suspeita
- Achado novo de balanceamento: só existe 1 rota de compra viável na Loja no início do jogo (documentado, não corrigido — é decisão de design, não bug)
 - **Autocorreção registrada:** 3 erros no meu próprio teste foram encontrados e corrigidos antes de virarem conclusão (argumento errado em `buyMachine`, nome de estado errado `'ANDAMENTO'` vs `'EM_ANDAMENTO'`, e uma falha de `localStorage` que era do ambiente de teste em origem opaca, não do jogo)

Não feito (registrado para não ser esquecido): A4 (integração cruzada), A5 (balanceamento completo), A7

(carreira longa formal), A8 (congelamento).

Arquivos gerados: AUDITORIA-FASE-A.md , teste-fase-a-financeiro.js , teste-fase-a6-bughunt.js , este REGISTRO.md .

Sessão 4 – 01/09/2026 (tarde/noite) – Sistema de compra de sede + registro do projeto

Pedido: sede não devia ser desbloqueio automático por patrimônio; devia ser **comprada**, exigindo missões cumpridas + reputação + dinheiro, com contador visível de quantas missões faltam.

Feito:

- Removido o desbloqueio automático (verificarDesbloqueioSedes)
- Criado sistema de campanhas: 4 campanhas (uma por sede-alvo), 17 missões no total, cada uma com progresso derivado do estado real do jogo (sem contador duplicado)
- Criada função statusCompraSede() que verifica os 3 requisitos e comprarSede() que executa a compra
- Tela de Missões reescrita: mostra a campanha atual, quantas faltam, e lista as missões com barra de progresso e botão de resgate
- Tela de Sedes reescrita: 3 barras de requisito + botão de compra que habilita só com os 3 verdes

- 44 testes automatizados escritos e passando (arquivo `teste-campanhas.js`)

Incidente: um script de edição automatizada apagou por acidente um bloco de código (`registrarHistorico` , `ajustarReputacao` , `let historico`) ao substituir uma função vizinha. O jogo quebrou no carregamento. Foi pego pelo teste automatizado antes da entrega, não pelo usuário — mas é o tipo de erro que motivou este registro.

Correção de rumo: o documento de status anterior dizia que "envelhecimento é cosmético". Isso estava errado — reauditei o código e confirmei que a condição da frota entra de verdade no cálculo de risco. Corrigido no STATUS-E-ROADMAP.

Arquivos gerados: `CAMPANHAS-DE-SEDE.md` , `teste-campanhas.js` , este `REGISTRO.md` .

Sessão 3 — 01/09/2026 (tarde) — Organização do projeto + correção de Sedes/Missões

Contexto recebido: usuário reportou que Sedes não mostrava imagem/objetivos e Missões não aparecia, mesmo após entregas anteriores dizerem que estava pronto.

Causa raiz encontrada (rodando o jogo de verdade em navegador headless, não por inspeção visual do código):

- `SEDES_DATA.imagem` era uma cor hexadecimal, não uma foto

- `abrirOficinas()` existia mas **nenhum botão a chamava** — sem acesso à oficina, fidelidade ficava sempre vazia
- Tela de Missões ignorava o array `missoes` que já existia e funcionava no Hub
- Oficina descontava dinheiro mas não consertava a máquina
- Formatação de moeda duplicada (R\$ R\$)

Feito:

- Adicionado botão "🔧 Fazer em Oficina Parceira" na tela de Manutenção
- Oficina agora recupera a condição da máquina de verdade
- Sedes com fotos reais (Unsplash), níveis bloqueados em preto e branco
- Tela de Missões reescrita mostrando as missões do Hub + fidelidade de marca
- **Reorganização de arquivos:** de 7 versões de `app.html` e 32 documentos soltos, para estrutura `01-JOGO/ 02-DADOS/ 03-PLANO/ _arquivo/`
- 27 testes automatizados escritos (arquivo `teste-automatizado.js`)

Causa raiz do problema maior (documentada no STATUS-E-ROADMAP): o projeto não tinha fonte única de verdade. Cada correção ia para uma cópia diferente do arquivo.

Sessão 2 – 01/09/2026 (manhã/tarde) – Marketing e vídeo promocional

Criação de pitch deck para investidores (PowerPoint, 12 slides), prompts para geração de vídeo via Kling AI, estratégia de marketing multi-plataforma. Kling ficou pago no meio da sessão; foram fornecidas alternativas gratuitas (Runway ML, CapCut, Synthesia). Vídeo não foi gerado com sucesso por nenhuma ferramenta.

*(Detalhes completos arquivados em
`_arquivo/marketing/.`)*

Sessão 1 – 31/08 a 01/09/2026 – Implementação original de Sedes/Oficinas/Missões

Criação do roadmap de 12 fases (depois consolidado em A–G), desenho do sistema de Sedes (5 níveis), Oficinas Parceiras (6 tipos/marcas), Missões de fidelidade, contratos Tier 2/3, whitepaper de monetização. Upload para GitHub Pages. Início da implementação em `app.html` – que viria a ser a implementação nunca testada que causou os problemas da Sessão 3.

*(Detalhes completos nas transcrições em
`/mnt/transcripts/.`)*

3. PRÓXIMO PASSO

Onde as coisas estão de verdade:

-  Fase 0-2 do roadmap de game feel – completas (sessões 14-29)
-  Fase 3, item 9 (sessão 30): identidade das máquinas – apelido, histórico, selo de veterana
-  Fase 3, item 11 (sessão 34): Consultor – conselhos reais baseados em estado do jogo, com moderação
-  Primeiro playtest humano real recebido e tratado (sessão 34): 30 minutos de jogo trouxeram 3 achados concretos, todos investigados e corrigidos na mesma sessão – travamento de contrato por falta de máquina (sorteio agora ponderado 80/20), missões recalibradas (curva inteira, não só remendo), e o Consultor construído com a especificação exata que o usuário deu

Decisões de design que ficaram pendentes, ainda sem resposta do usuário:

1. **Da sessão 23/24:** todo evento de risco garante atraso 100% das vezes, mesmo com a opção mais cara de engenharia. Pergunta: pagar mais deveria reduzir também o `extraDias` ?
2. **Da sessão 5:** achado de balanceamento da Loja (nenhuma máquina compra à vista com o caixa inicial).

Resolvido, não é mais pendência: a dúvida da sessão 21 sobre aceitar contrato sem máquina certa – o jogo já

bloqueia isso.

- ☐ Fase 3, item 10: marcos e conquistas da empresa
- ☐ Fase 3, item 12: diário de notícias mais vivo — parcialmente já atendido pelo Consultor (sessão 34), vale revisar se ainda falta algo específico do diário em si antes de investir mais tempo nisso
- ☐ Fase 4 do roadmap — não iniciada (som, transições, identidade visual)
- ☐ A4/A5/A7/A8 da Fase A original — seguem soltas, sem bloqueio

Nota técnica que vale carregar pra próxima sessão: a fórmula de prazo recalibrada (sessões 22-24) fez os contratos ficarem mais curtos e previsíveis — isso é bom pro jogo, mas quebra a suposição de testes que assumem "muitas tentativas = quase certeza estatística" pra eventos raros dentro de 1 contrato só (achado nas sessões 30 e continua valendo). Testes desse tipo precisam de retry através de múltiplos contratos, não confiar só no tempo de vida de 1 único.

Recomendação: a pausa que vinha sendo sugerida há várias sessões **já aconteceu na prática** — o usuário trouxe playtest real, não precisou de um pedido formal. Isso valida que o padrão "jogar organicamente e trazer o que incomodou" está funcionando melhor que esperar o preenchimento do `GUIA-PLAYTEST.md`. Sugiro manter esse ritmo: seguir pra próxima peça do roadmap (item 10 ou 12) até o próximo playtest real trazer o próximo ajuste de rota.

4. REGRAS DESTE REGISTRO (para mim mesmo, em sessões futuras)

1. **Nunca apagar histórico.** Só adicionar sessão nova no topo da seção 2.
2. **Seção 1 sempre é reescrita**, não acumulada — ela é a fotografia do presente.
3. Antes de editar `app.html` com scripts automatizados, **rodar teste depois, nunca confiar na edição sem executar o arquivo de verdade** — foi exatamente a falta disso que causou o incidente da Sessão 4.
4. Toda sessão que altera o jogo termina com: teste automatizado rodando, arquivo `app.html` atualizado em `01-JOG0/`, e uma entrada nova aqui.
5. Se um arquivo de teste (`teste-*.js`) for descartado, ele vai para `_arquivo/`, nunca é deletado — serve de prova do que foi validado.